S

SmartPOS Application Design

The whole application explained: how it is built, how the screens work, how the database is designed, every validation rule, and the logic of each feature, with flowcharts.

Architecture | App design | Database design | Validation | Logic | Flowcharts

---

Angular 22 web app · ASP.NET Core 10 API · EF Core · PostgreSQL 16 · local AI (Ollama, qwen3.5:9b)
Every screenshot, response, figure and database row in this document was captured from the running application on a throwaway database on 4 November 2026.

Contents

Part A · The big picture

1. What SmartPOS does
2. Architecture
3. How the web app is built
4. Security and shop isolation

Part B · Database design

5. The data model
6. Every table

Part C · Validation

7. How validation works
8. Every rule, form by form

Part D · Features

9. Accounts: register, sign in, verify
10. Dashboard
11. Products and categories
12. Point of Sale and checkout
13. Sales history and invoices
14. Customers and loyalty points
15. Inventory and stock movements
16. Reports
17. AI assistant

Part E · Reference

18. API endpoints
19. Settings
20. Things to know and limitations
21. How this document was checked

PART A

The big picture

What the application does, the pieces it is made of, how a request travels through them, and how each shop's data is kept separate.

1 What SmartPOS does

SmartPOS is a point-of-sale and inventory system for small shops: a grocery, a pharmacy, a general store. The shop owner signs up, which creates their account **and** their shop. From then on they can:

Area	What the owner can do	Screen
Point of Sale	Ring up a bill: search or scan products, attach a customer, give a discount, take Cash/Card/UPI/Other, see change due, print the bill	/pos
Products	Keep the catalogue: SKU, barcode, prices, category, expiry date, active or not	/products
Inventory	See stock levels (most urgent first), record deliveries and write-offs, read the full stock history	/inventory
Customers	Keep regulars by phone number; they earn loyalty points on every bill	/customers
Sales	Look up any bill by date or number and reprint it	/sales
Dashboard	Today's figures, plus 7/30/90-day performance: revenue, bills, profit, busiest hours, top products and customers	/dashboard
Reports	Daily sales and top products for any date range	/reports
AI assistant	Ask questions in plain language ("Which products are low on stock?"), answered from the shop's own data by a model running on the same computer	/assistant

THREE IDEAS THAT RUN THROUGH EVERYTHING

- **Many shops, one database.** Every row belongs to a shop, and a signed-in owner can only ever see their own shop's rows.
- **The server is the authority.** The browser checks things to be helpful, but prices, totals, stock and permissions are always decided on the server.
- **Nothing is half-saved.** A sale (stock, bill, lines, ledger, loyalty points) is saved completely or not at all.

2 Architecture

2.1 The system

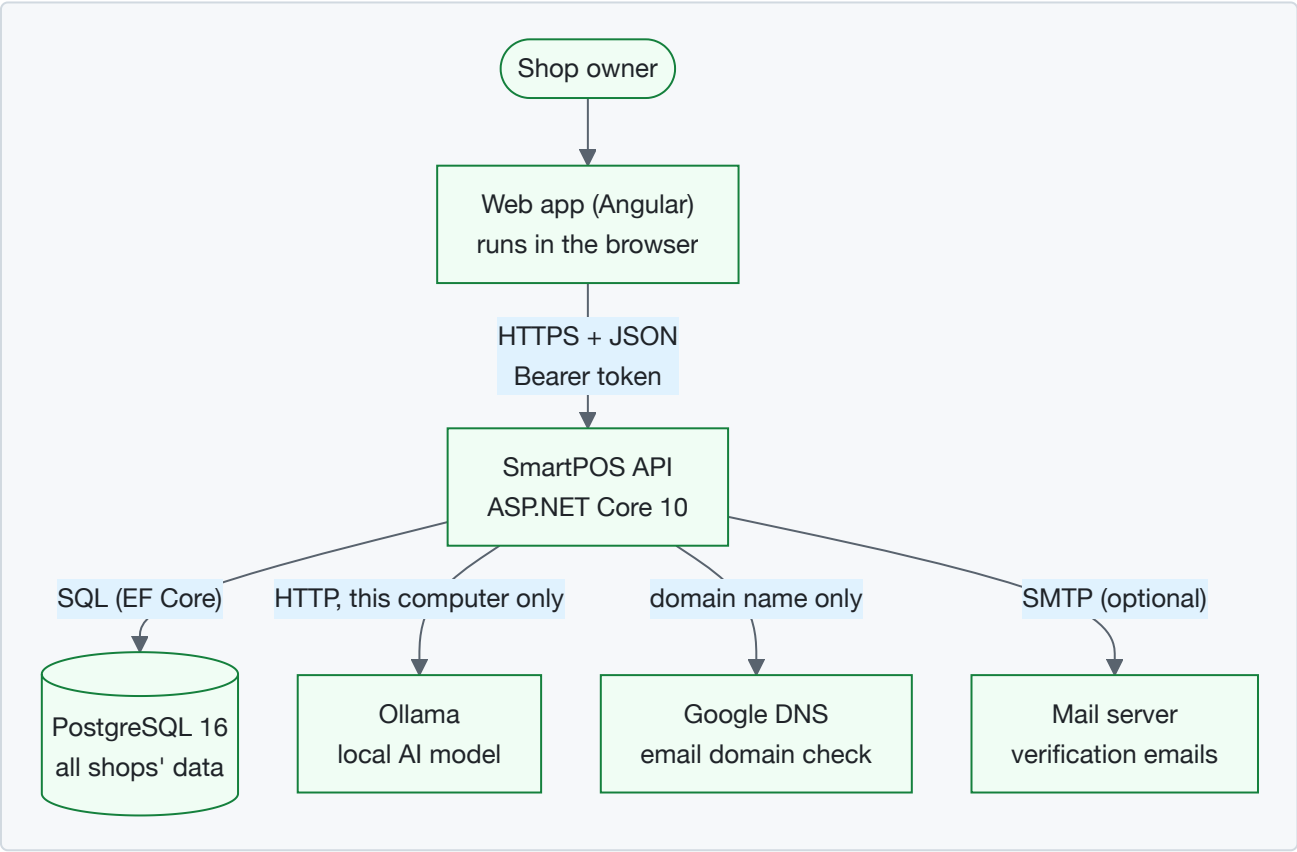


Figure 2.1. The pieces and how they talk. The browser only ever talks to the API.

2.2 The back end: four layers

The API is split into four projects. Each one may only use the layers *below* it, so the business rules don't depend on the web or the database.

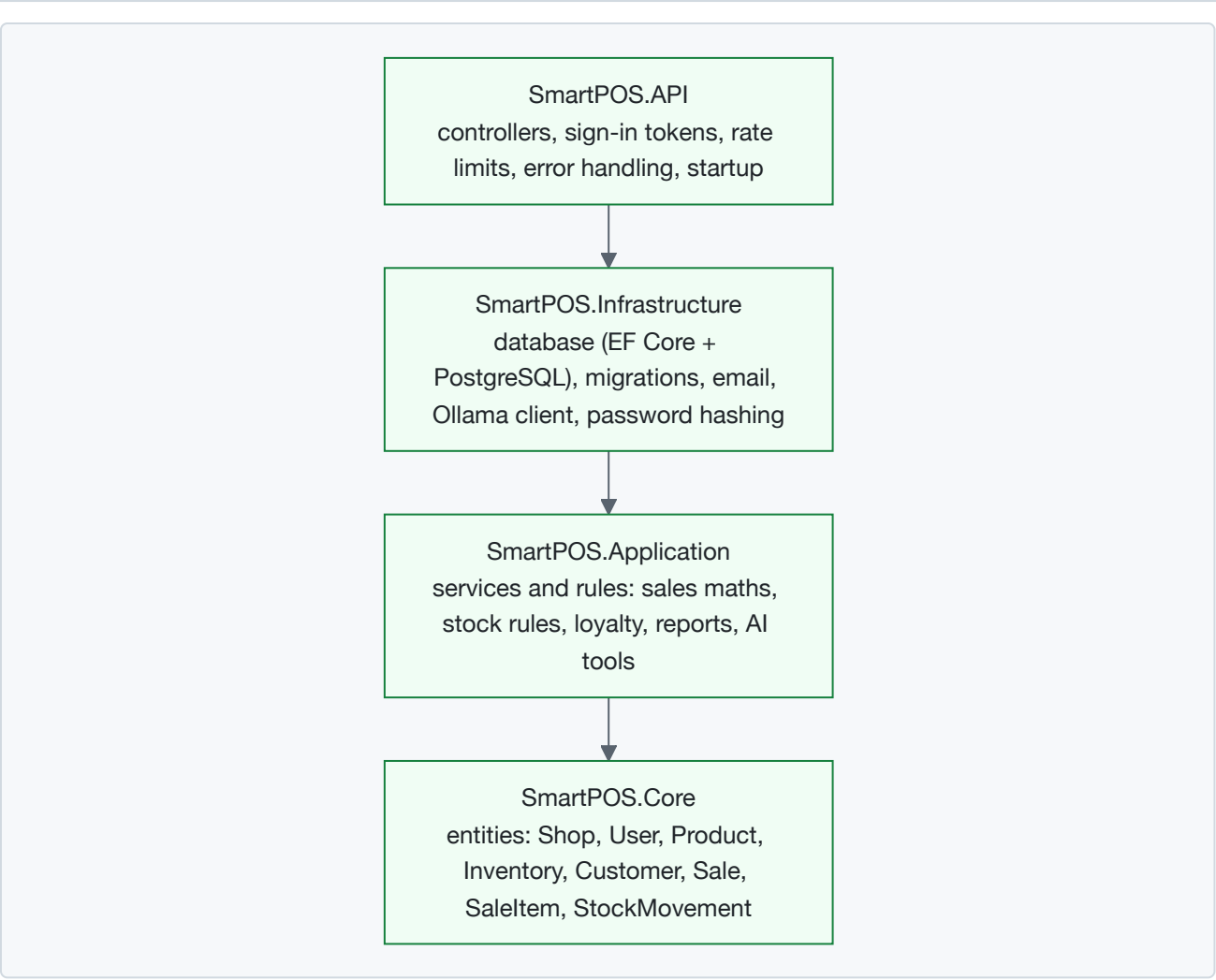


Figure 2.2. Layers ("Clean Architecture"). The arrows mean "depends on".

Layer	Holds	Example
Core	The data model only, with no logic	`Sale`, `SaleItem`
Application	What the app *does*: services, rules, request/response shapes. It declares interfaces for what it needs (database, password hasher, email, AI model).	`SaleService`, `SaleCalculator`, `StockRules`
Infrastructure	Talks to the outside world, implementing those interfaces	`SmartPosDbContext`, `OllamaChatClient`
API	The web server: receives HTTP, checks the token, calls one service method, turns errors into HTTP responses	`SalesController`, `GlobalExceptionHandler`

2.3 What happens to every request

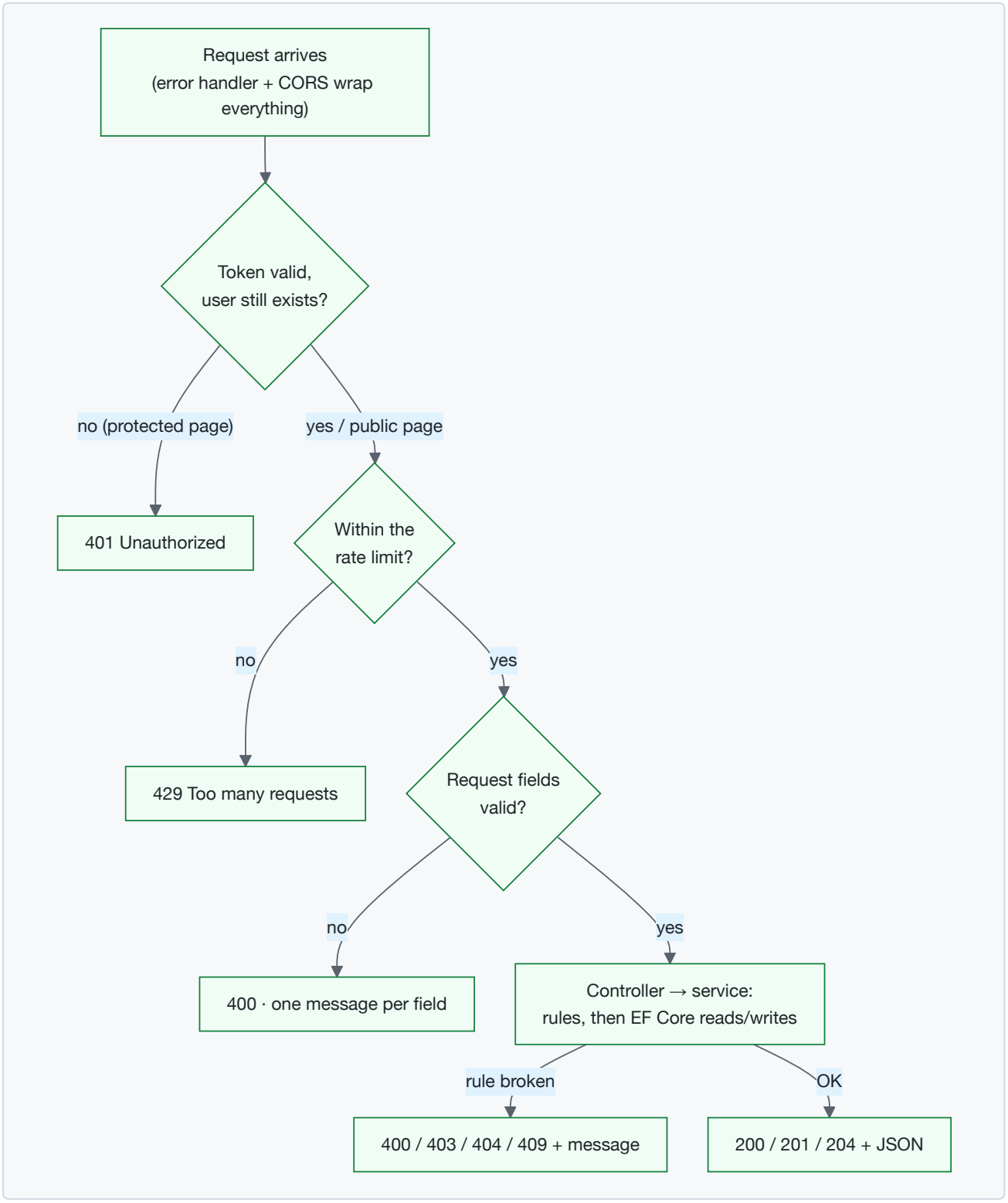


Figure 2.3. The request pipeline, set up in Program.cs

3 How the web app is built

The web app is an Angular 22 single-page app. The browser downloads it once and then only exchanges JSON with the API. Each page is a **standalone component** that is loaded the first time you open it. Screen state is kept in **signals**, and forms are **reactive forms** with their rules written in code.

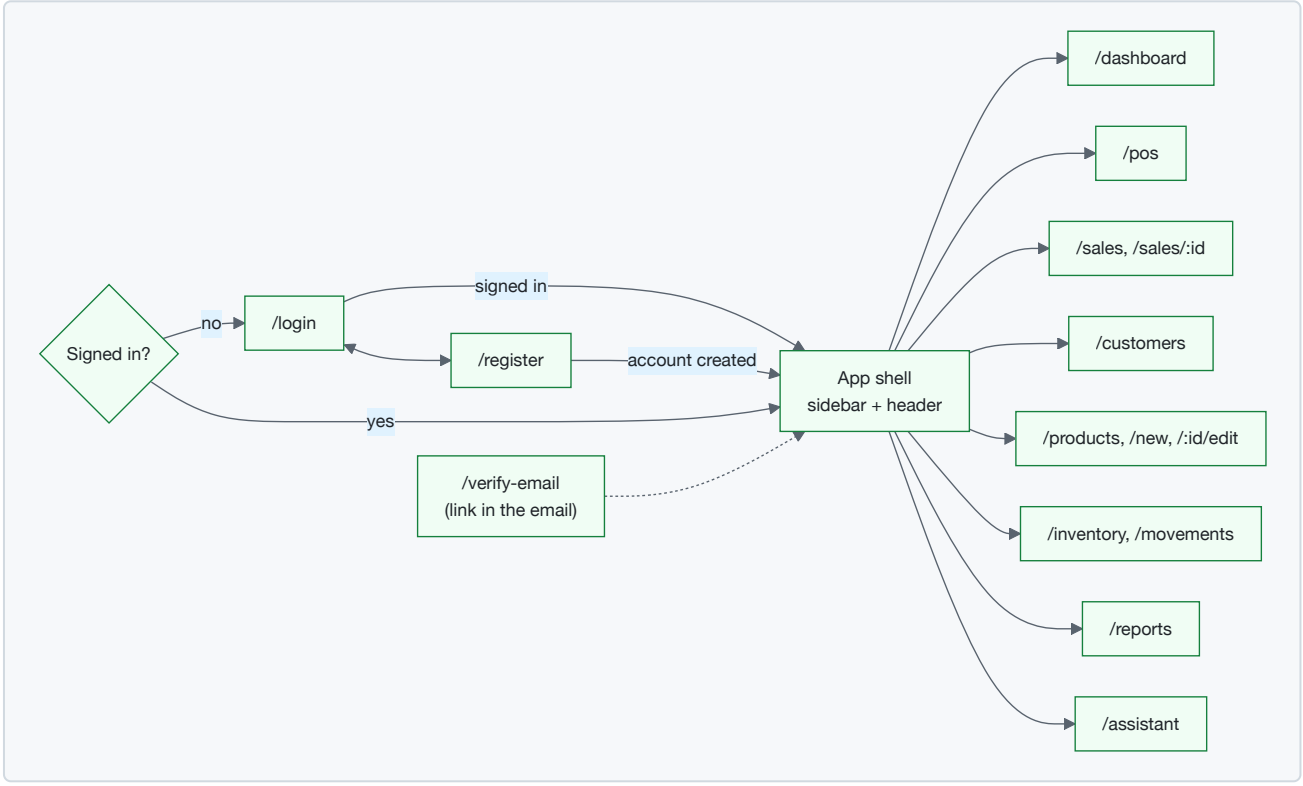


Figure 3.1. Pages and navigation

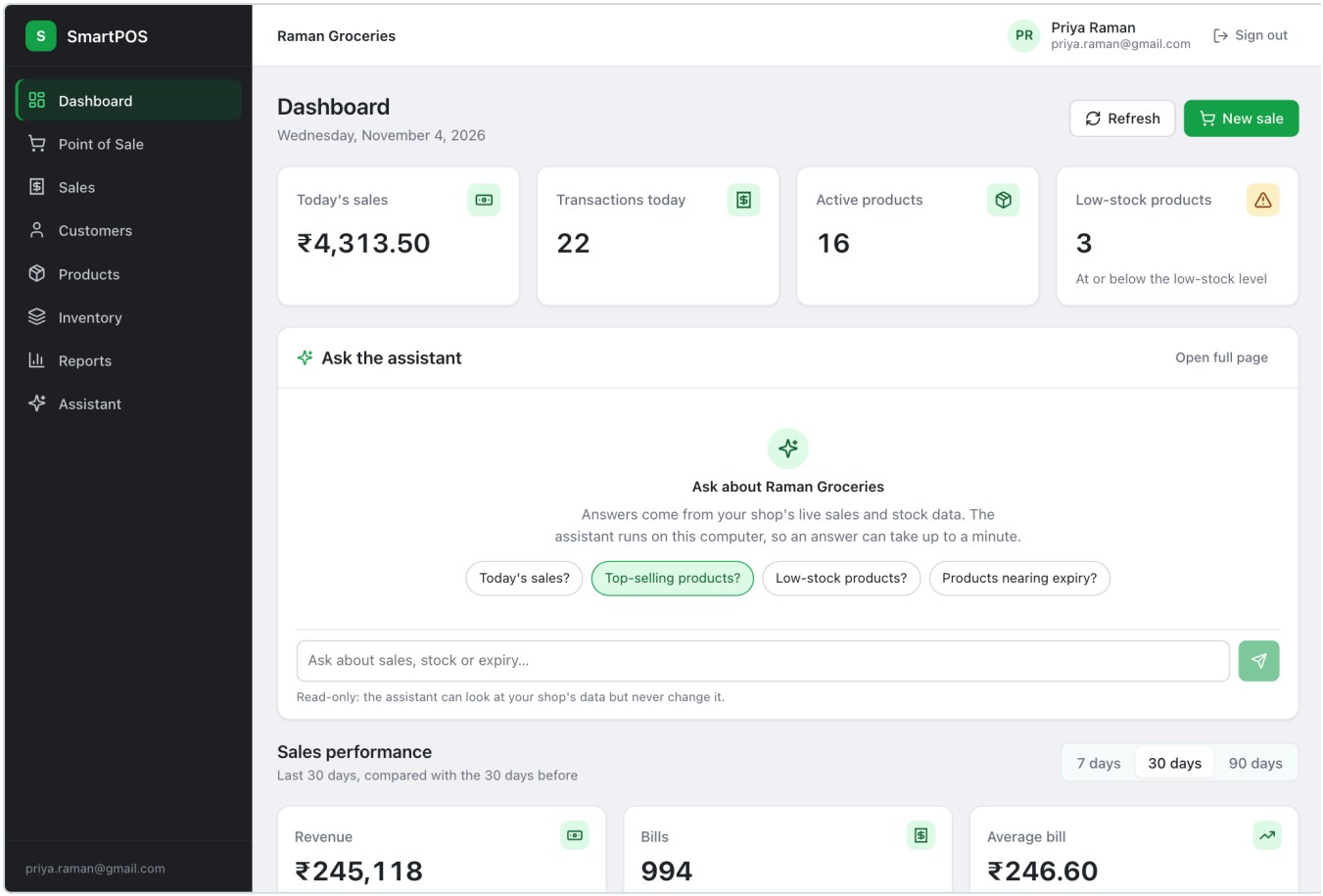


Figure 3.2. The app shell: the sidebar to the left, the shop name and user at the top, the page in the middle. This is the dashboard of the test shop "Raman Groceries".

Folder (frontend/src/app)	What lives there
`core/api/`	One service per API area (products, sales, customers, inventory, settings, assistant). Pages never build URLs themselves.
`core/auth/`	The session (token kept in browser storage), the route guards, and the **interceptor** that adds `Authorization: Bearer …` to every API call and signs you out on a 401
`core/http/`	`errorMessage()`, which turns any error response into one sentence for the screen
`core/models.ts`	TypeScript copies of every API request and response shape
`features/…`	One folder per area: auth, dashboard, pos, products, customers, inventory, sales, reports, assistant
`layout/`	Shell, sidebar, header
`shared/`	Reusable pieces: icons, modal dialog, toast, pagination, empty/error/loading states, dependency-free SVG charts, the Markdown renderer for the assistant

COMMON SCREEN BEHAVIOUR

- **Loading, empty and error states** exist on every list ("No sales yet", "Could not load…" with a Retry button).
- **Form errors** appear under a field once it has been touched, or all at once when you press Save. An invalid form is never sent.
- **Server errors** appear as a red alert with the server's own message; **success** shows a short toast.
- **Destructive actions** (clear the cart, delete a product) ask for confirmation first.

4 Security and shop isolation

4.1 Signing in and staying signed in

- Signing in returns a **JWT** (a signed token) valid for **8 hours**. It contains the user's id, name, email and **shop id**, and is signed with a secret key only the server knows, so it can't be edited.
- On every request the server checks the signature and expiry, then checks in the database that the user still exists **in that shop**.
- Every endpoint requires a signed-in user unless it is explicitly public. The only public ones are sign-in, register, verify-email and `/health`.
- Passwords are stored only as **PBKDF2** hashes (100,000 rounds, a random salt each). A wrong password and an unknown email get the **same** message, in about the same time.
- **Rate limits:** 10 sign-in/register attempts per minute per IP address, and 10 AI questions per minute per user.

4.2 How each shop only sees its own data

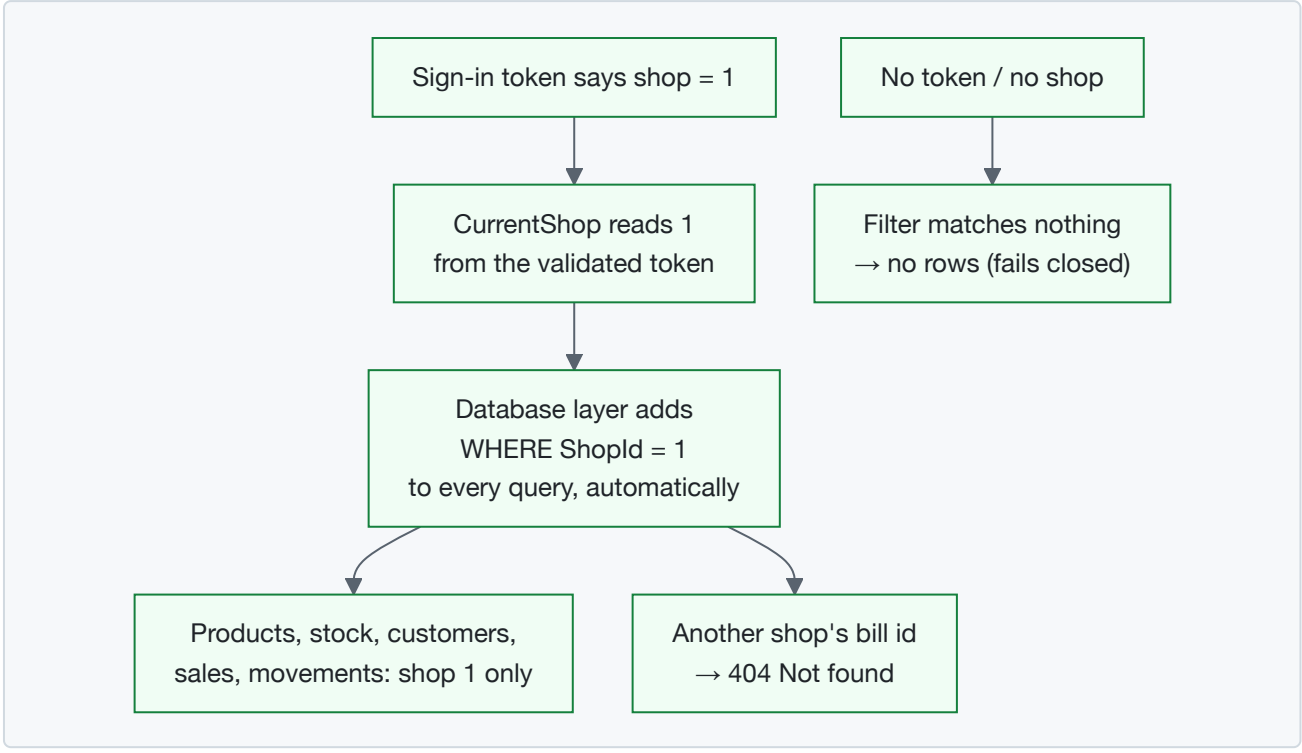


Figure 4.1. Shop isolation (an EF Core "global query filter")

Because the filter sits in the database layer, a developer can't forget it, and the AI assistant's tools go through it too. Tested for this document: shop 2 asking for shop 1's bill got **404 "Sale 5756 was not found."**, and shop 2 trying to sell shop 1's product got **400 "Product 1 does not exist or is not available for sale."** New rows always take their shop from the token, never from the request.

PART B

Database design

Eight tables in PostgreSQL, created and changed only through EF Core migrations. What each table holds, how they connect, and the rules the database itself enforces.

5 The data model

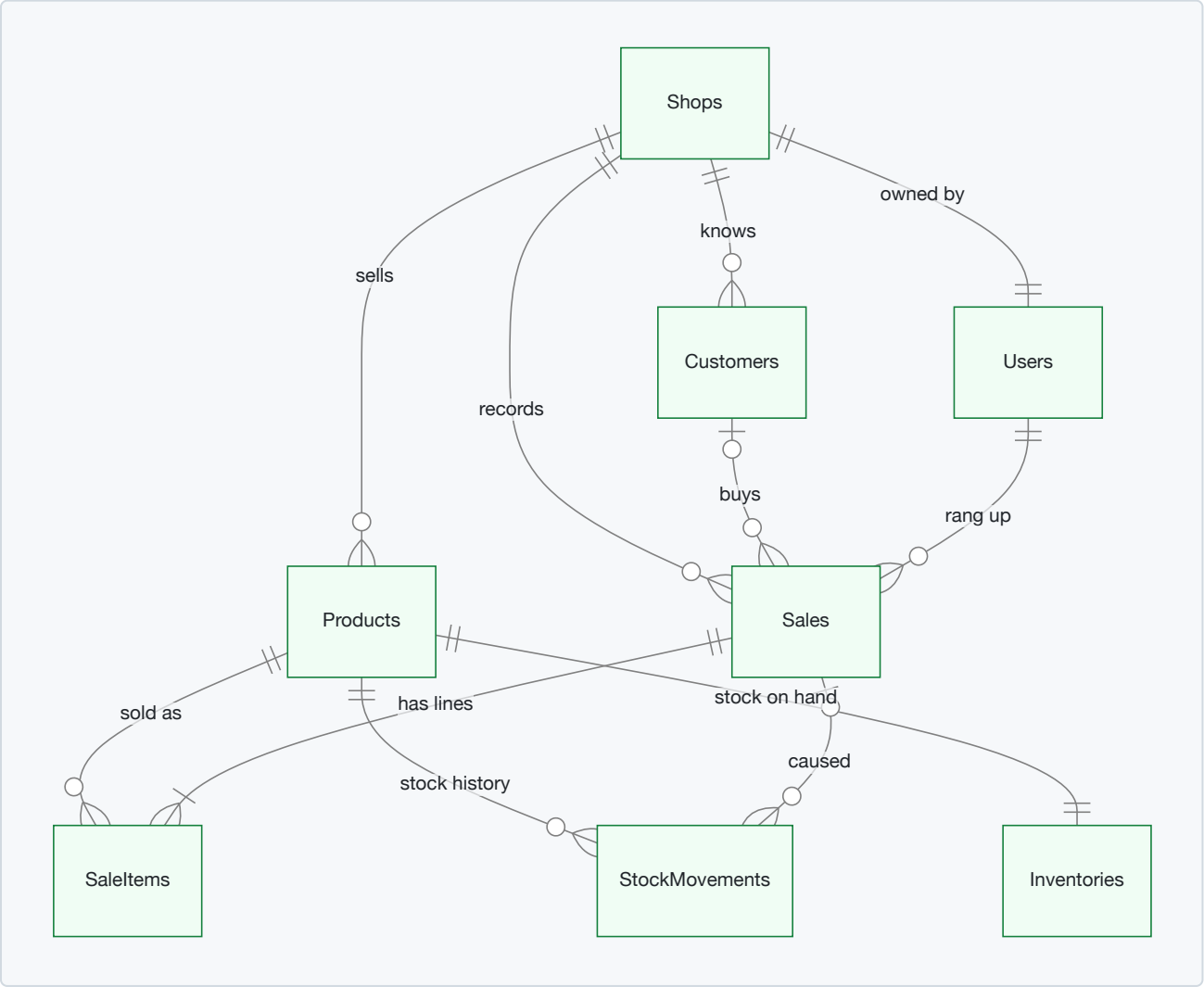


Figure 5.1. The eight tables and how they relate (every column is listed in section 6)

5.1 Design decisions

Decision	Why
Every business table has a `ShopId` (sale lines inherit it from their sale)	Many shops share one database safely; the automatic shop filter needs it
One owner per shop: `Users.ShopId` is unique	Registering creates the user and the shop together; there are no staff accounts or roles
Stock in its own table (`Inventories`), not on the product	Editing a product can never change stock; stock changes only through sales and adjustments
A stock ledger (`StockMovements`): one row per change, never edited	Current stock can always be explained; replaying the movements gives exactly the stock on hand
Snapshots on sale lines: product name, SKU, price and cost are copied onto `SaleItems`	Renaming or repricing a product later never changes an old bill or an old profit report
Money as `numeric(12,2)` / `(18,2)`, never floating point	0.10 is exact; rounding is half away from zero, as on a receipt

Decision	Why
Timestamps in UTC; "today" is worked out in the shop's time zone (Asia/Kolkata)	A sale at 1 am IST still counts for the right day
Enums stored as text (`'Cash'`, `'Completed'`, `'Sale'`)	The tables are readable without a code lookup
Category is free text on the product, not a table	A new category exists as soon as a product uses it
CHECK constraints on stock, prices, totals, quantities, points	Even a bug in the code can't store negative stock or a negative total
Deleting is restricted wherever history depends on the row	A sold product, a customer with bills, a shop with data can't be deleted out from under their history

5.2 Migrations

Migration	What it did
`20261028092049_InitialCreate`	Created all eight tables, keys, indexes and checks
`20261028111930_AddLoyaltyAndEmailVerification`	Added email-verification columns on `Users` and `LoyaltyPointsEarned` on `Sales`, then back-filled points for existing bills
`20261103131345_RemoveTax`	Recalculated every bill without tax, then dropped the three tax columns

6 Every table

Columns as they exist in PostgreSQL. **?** means the column may be empty (NULL). **PK** = primary key, **FK** = foreign key.

Shops: one row per shop

Column	Type	Notes
Id	integer PK	generated
ShopName	varchar(150)	
Address	varchar(250)	empty string if not given
CreatedAt	timestamptz	

Users: the shop owner's login

Column	Type	Notes
Id	integer PK	
ShopId	integer FK → Shops	**unique**: one owner per shop
FullName	varchar(100)	
Email	varchar(254)	**unique**, stored lower-case
PasswordHash	varchar(500)	PBKDF2 hash; the password itself is never stored
EmailVerifiedAt / EmailVerificationTokenHash / EmailVerificationExpiresAt	timestamptz? / varchar(64)? / timestamptz?	only a SHA-256 hash of the emailed secret is stored
CreatedAt, UpdatedAt, LastLoginAt?	timestamptz	

Products: the catalogue

Column	Type	Notes
Id, ShopId	integer PK, FK → Shops	
Sku	varchar(50)	upper-case; unique per shop
Barcode	varchar(50)?	digits; unique per shop when present
Name / Description? / Category	varchar(150) / (1000) / (100)	
Price, CostPrice	numeric(18,2)	CHECK ≥ 0
ExpiryDate	date?	
IsActive	boolean	inactive = hidden from the POS, history kept
CreatedAt, UpdatedAt	timestamptz	

Inventories: stock on hand (exactly one row per product)

Column	Type	Notes
Id, ShopId, ProductId	integer PK, FK, FK → Products (**unique**, delete cascades)	
StockQuantity	integer	**CHECK ≥ 0**: stock can never be negative

Column	Type	Notes
LowStockThreshold	integer	CHECK ≥ 0; "low" means at or below this
LastUpdated	timestamptz	

StockMovements: the stock ledger

Column	Type	Notes
Id, ShopId, ProductId	integer PK, FK, FK	
QuantityChange	integer	+ in (opening stock, delivery), – out (sale, damage)
QuantityAfter	integer	CHECK ≥ 0; stock right after this change
Type	varchar(20)	InitialStock / Sale / Adjustment
Reason	varchar(250)	"Opening stock", "Sale BILL-…", "Delivery from supplier"
SaleId?, UserId?	FK → Sales, FK → Users	which bill, and who
CreatedAt	timestamptz	

Customers

Column	Type	Notes
Id, ShopId	integer PK, FK	
Name	varchar(100)	
Phone	varchar(20)	stored as digits ("98400 12345" → "9840012345"); **unique per shop**
LoyaltyPoints	integer	CHECK ≥ 0; always the sum of its bills' points
CreatedAt	timestamptz	

Sales: one row per bill

Column	Type	Notes
Id, ShopId	integer PK, FK	index (ShopId, CreatedAt) for date filters
BillNumber	varchar(50)	**unique**, e.g. BILL-20261104-89134635
UserId, CustomerId?	FK → Users, FK → Customers	who rang it up; optional customer (a customer with bills can't be deleted)
Subtotal, DiscountAmount, TotalAmount	numeric(12,2)	total = subtotal – discount; CHECK ≥ 0
DiscountPercent	numeric(5,2)	CHECK 0–100
PaymentMethod, Status	varchar(30), varchar(20)	Cash/Card/UPI/Other; Completed (Pending/Cancelled reserved)
AmountPaid, ChangeAmount	numeric(12,2)	CHECK ≥ 0; change only for cash
LoyaltyPointsEarned	integer	CHECK ≥ 0
CreatedAt	timestamptz	

SaleItems: the lines of a bill

Column	Type	Notes
Id, SaleId	integer PK, FK → Sales (cascade)	
ProductId	FK → Products (**restrict**)	a product with sales can't be deleted, only deactivated
ProductName, Sku	varchar(200), varchar(100)	**snapshot** at the time of sale
Quantity	integer	CHECK > 0
UnitPrice, UnitCost	numeric(12,2)	snapshots; cost feeds the profit reports
DiscountAmount, LineTotal	numeric(12,2)	this line's share of the discount; lines add up exactly to the bill

PART C

Validation

Every value is checked at up to four levels, each with a different job. Then every rule, form by form, with the real messages.

7 How validation works

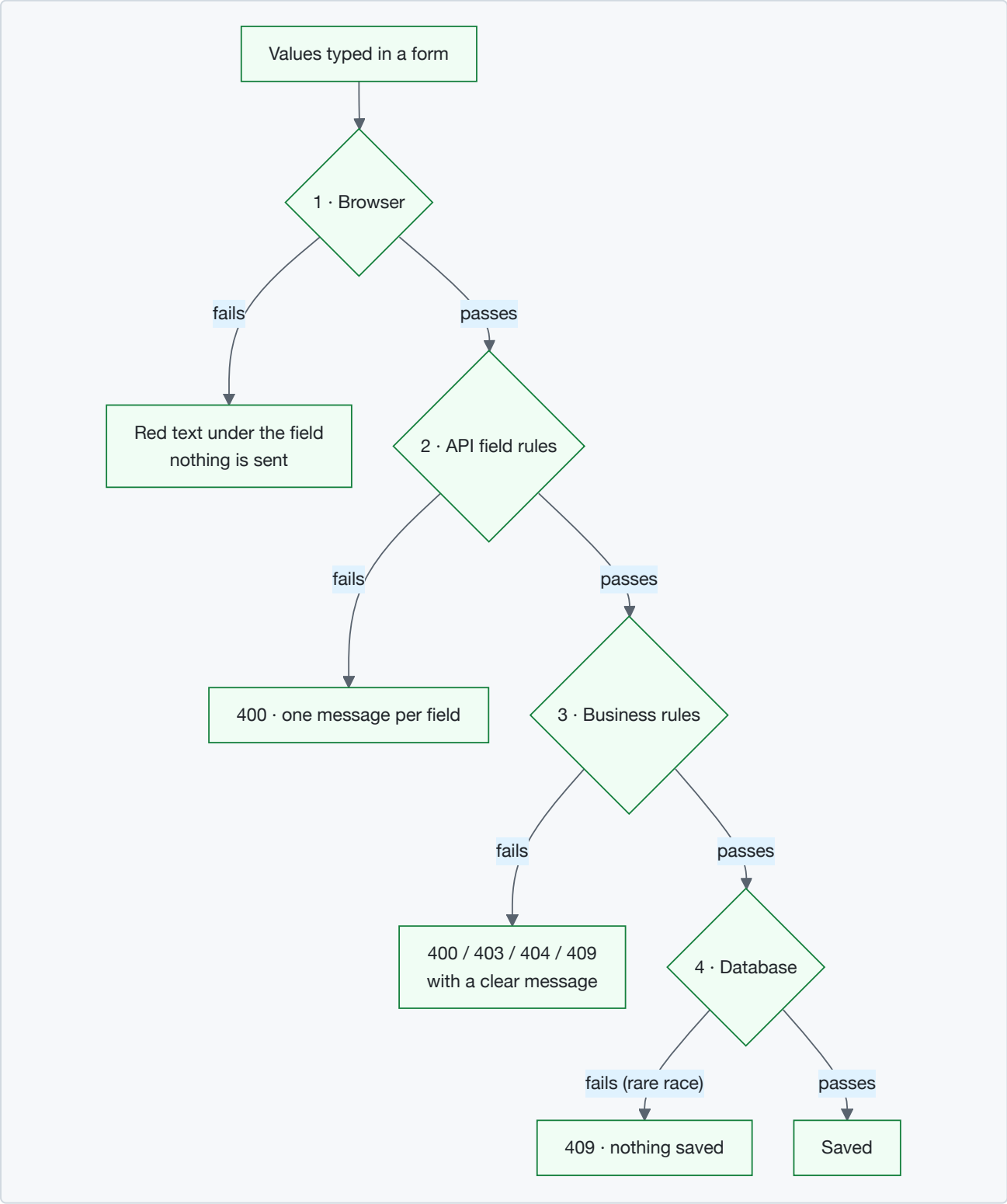


Figure 7.1. The four layers

Layer	Where	What it checks	Why it exists
1 · Browser	Angular form validators	Required, lengths, patterns, ranges, decimals	Instant, friendly feedback; saves a round trip

Layer	Where	What it checks	Why it exists
2 · API field rules	Attributes on request classes (`[Required]` , `[StringLength]` , `[Range]` , `[RegularExpression]` , `[MaxDecimalPlaces]`), checked automatically before any code runs	The same shape rules again	Requests can come from anywhere, so the browser is never trusted
3 · Business rules	Services and rule classes	Duplicates, stock, discount limit, payment, date ranges, ownership	Rules that need the database or the configuration
4 · Database	Unique indexes, CHECK constraints, foreign keys, NOT NULL, lengths	No duplicates, no negatives, no orphans	The last line of defence, even against bugs and races

7.1 How a failure becomes a message

Thrown by the code	HTTP	Example message (real)
field rule failed (automatic)	400	"Password must be 8-128 characters and contain at least one letter and one digit."
`BusinessRuleException`	400	"Amount paid (10.00) is less than the total (40.00)."
`AuthenticationFailedException`	401	"Invalid email or password."
`ForbiddenException`	403	"A discount can be at most 50%." (when `Pos:MaxDiscountPercent` is 50)
`NotFoundException`	404	"Sale 5756 was not found."
`ConflictException`	409	"Insufficient stock for 'Curd 400 g': requested 1, available 0."
database unique/constraint violation	409	"The change conflicts with existing data. Reload and try again."
`TooManyRequestsException` / rate limiter	429	"Your previous question is still being answered…"
`AIUnavailableException`	503	"The AI assistant is offline. Start Ollama and try again."
anything unexpected	500	"An unexpected error occurred." (details go to the server log only)

Every error uses the standard **problem details** JSON (`title` , `status` , `detail` or `errors`). The web app reads it in one place, `errorMessage()` , and shows the message in a red alert.

8 Every rule, form by form

Browser and API rules are the same unless noted. "Server" means business rules in the service.

Register (/register → POST /api/auth/register)

Field	Browser + API	Server
Your name	required, 2–100	trimmed
Email	required, valid email, ≤ 254	lower-cased; must not already have an account (409); disposable-mailbox domains refused (400); the domain must be able to receive mail (DNS check; a failed lookup doesn't block)
Password	8–128 characters with at least one letter and one digit	stored as a PBKDF2 hash
Shop name	required, 2–150	trimmed
Shop address	optional, ≤ 250	trimmed

Sign in (/login → POST /api/auth/login)

Field	Browser + API	Server
Email	required, valid email	unknown email or wrong password → the same 401; 10 attempts per minute per IP
Password	required (API ≤ 128)	

Product (/products/new, /products/:id/edit → POST / PUT /api/products)

Field	Browser + API	Server / database
SKU	required, 2–50, letters/digits/`.` / `_` / `-`, starts with a letter or digit	upper-cased; unique per shop (409)
Barcode	optional, digits only, ≤ 50	unique per shop when given (409)
Name / Category / Description	2–150 / 2–100 / ≤ 1000	trimmed; blank description → null
Selling price / Cost price	0 – 9,999,999.99, at most 2 decimals	CHECK ≥ 0
Opening stock (create only)	whole number 0 – 1,000,000	> 0 writes an "Opening stock" movement
Low-stock level	whole number 0 – 1,000,000 (default 20)	CHECK ≥ 0
Delete	asks for confirmation	only if never sold, else 409 "…Deactivate it instead."

Customer (/customers → POST / PUT /api/customers)

Field	Browser + API	Server / database
Name	required, 2–100	trimmed
Phone	7–20 characters: digits, spaces, `+ - ( )`, starting and ending with a digit	normalised to digits (+ kept); unique per shop (409 "A customer with phone '9840012345' already exists.")
Loyalty points	not editable	only ever changed by sales

Sale (/pos → POST /api/sales)

Field	Browser	API + server
Items	can't add more than in stock; quantity clamped to 1 – available	1–200 lines, quantity 1–10,000; product must exist in this shop and be active; enough stock (409 with the available count)
Discount %	0 – the configured maximum	0–100, ≤ 2 decimals; above `Pos:MaxDiscountPercent` → 403
Payment	cash received must cover the total	paid ≥ total (400); change only for cash (400 "Change can only be given for cash payments.")
Customer	picked from a search	must exist in this shop
Prices	shown as a preview only	**never accepted from the browser**; read from the database

Stock adjustment (/inventory → POST /api/inventory/adjustments)

Field	Browser	API + server
Change	Add or Remove, quantity 1–100,000; removing more than on hand is blocked ("Only 6 on hand; you cannot remove 10.")	−100,000 to +100,000, not 0 (400); result can't go below 0 (409)
Reason	required, 3–250	stored on the movement

Date ranges and queries

Where	Rule	Real message
Reports	from ≤ to; at most 366 days; default last 7 days	"'from' must be on or before 'to'." / "Date range cannot exceed 366 days."
Dashboard insights	1–180 days (the screen offers 7/30/90)	"Days must be between 1 and 180."
Top products	limit 1–50	"Limit must be between 1 and 50."
Every list	page ≥ 1, page size 1–100, search ≤ 100 characters (bill-number search ≤ 50)	400

AI question (/assistant → POST /api/ai/chat)

Field	Rule
Message	1–1,000 characters
History	at most 40 turns; each turn's role must be `user` or `assistant` (400 "Role must be 'user' or 'assistant'." for anything else), content 1–4,000
Rate	one question at a time per user (429), 10 per minute (429)

PART D

Features

Each feature: what the screen does, the flowchart of its logic, the rules that apply, and the tables it reads and writes.

- Accounts
- Dashboard
- Products
- Point of Sale
- Sales history
- Customers and loyalty
- Inventory
- Reports
- AI assistant

9 Accounts: register, sign in, verify

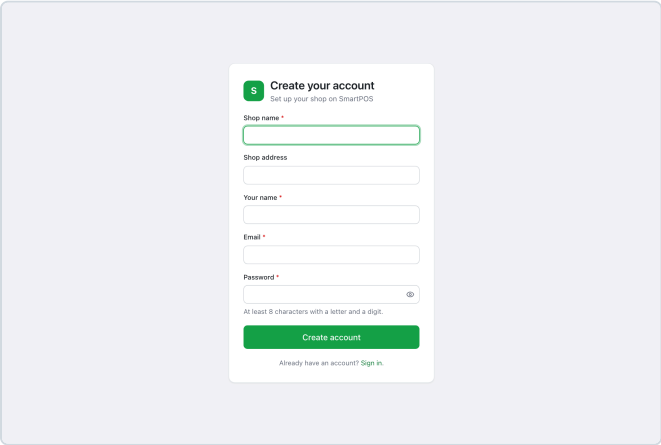


Figure 9.1. Create an account: shop name, address, your name, email, password

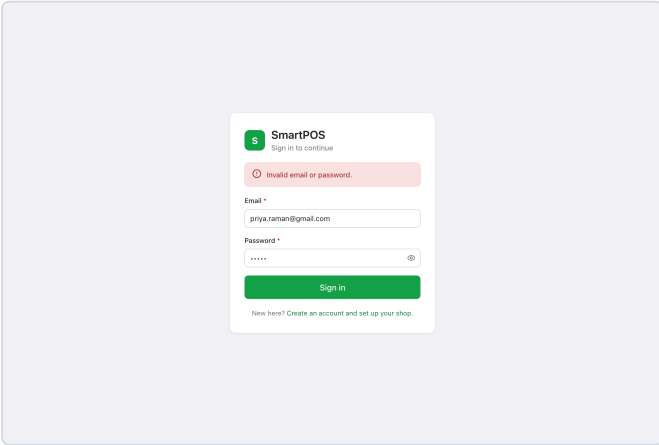


Figure 9.2. Sign in with a wrong password: "Invalid email or password."

9.1 Registering

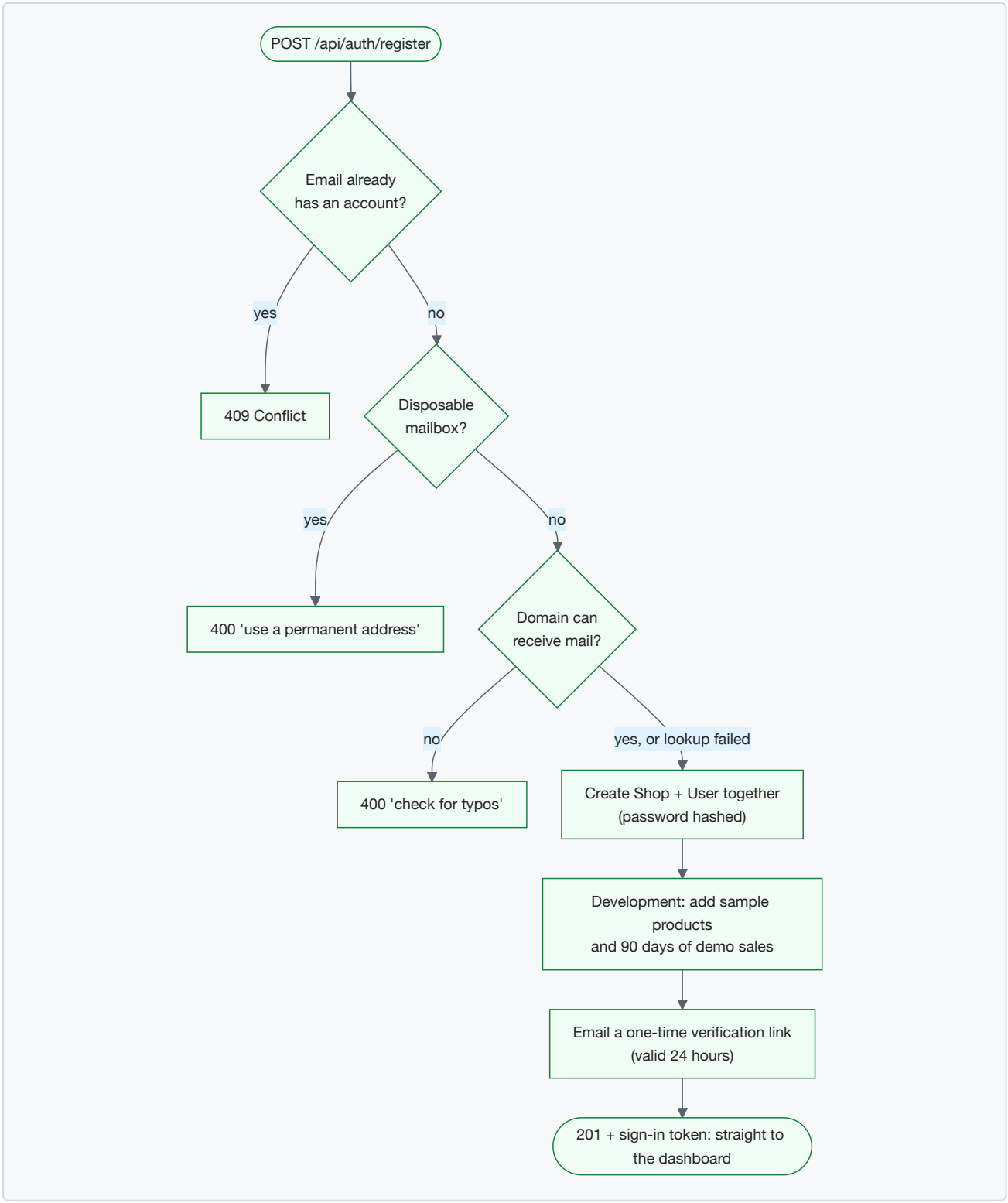


Figure 9.3. Registration (AuthService.RegisterAsync)

9.2 Signing in and signing out

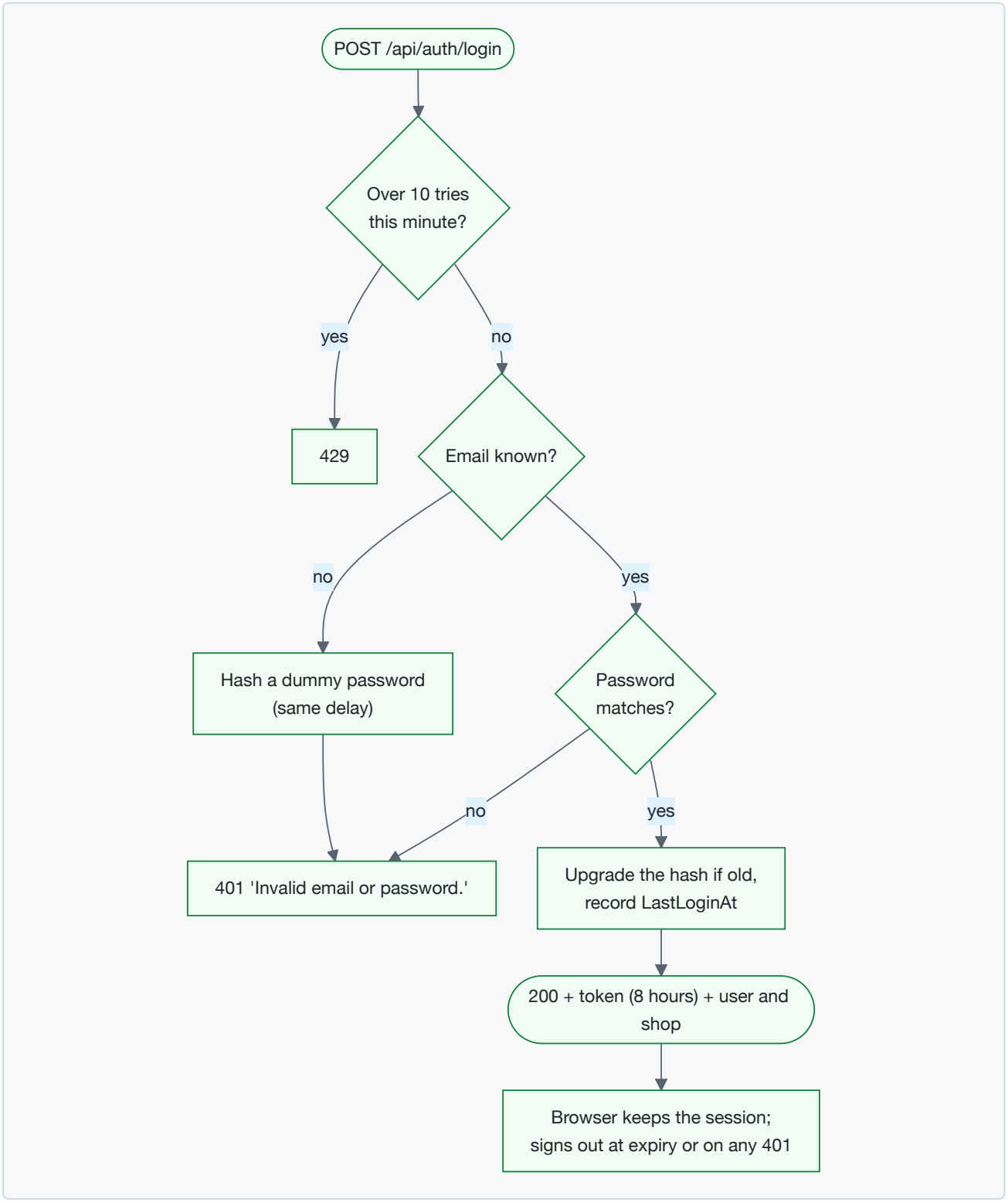


Figure 9.4. Sign-in (AuthService.LoginAsync)

9.3 Verifying the email address

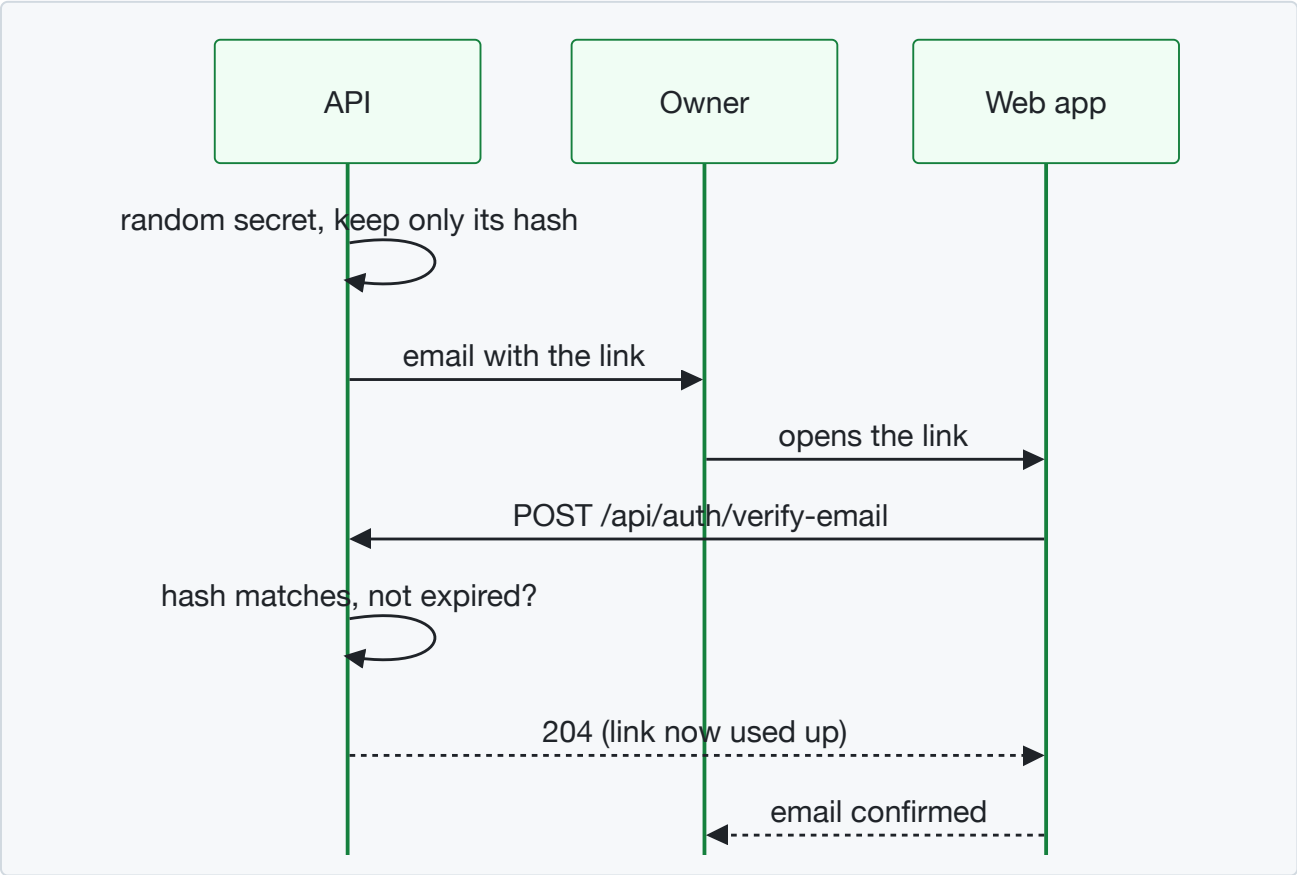


Figure 9.5. Email verification

Tested: the first use of the link gave **204**, and the second gave **400** "This verification link is invalid or has expired. Sign in and request a new one." Until the address is confirmed, a yellow banner offers "Resend link". Verification is **not enforced**: an unverified owner can use everything.

Tables: writes `Shops`, `Users` (and in Development `Products`, `Inventories`, `Customers`, `Sales`, `SaleItems`, `StockMovements`). **Code:** `AuthController`, `AuthService`, `IdentityPasswordHasher`, `JwtTokenService`, `EmailServices`; web `features/auth/*`, `core/auth/*`.

10 Dashboard

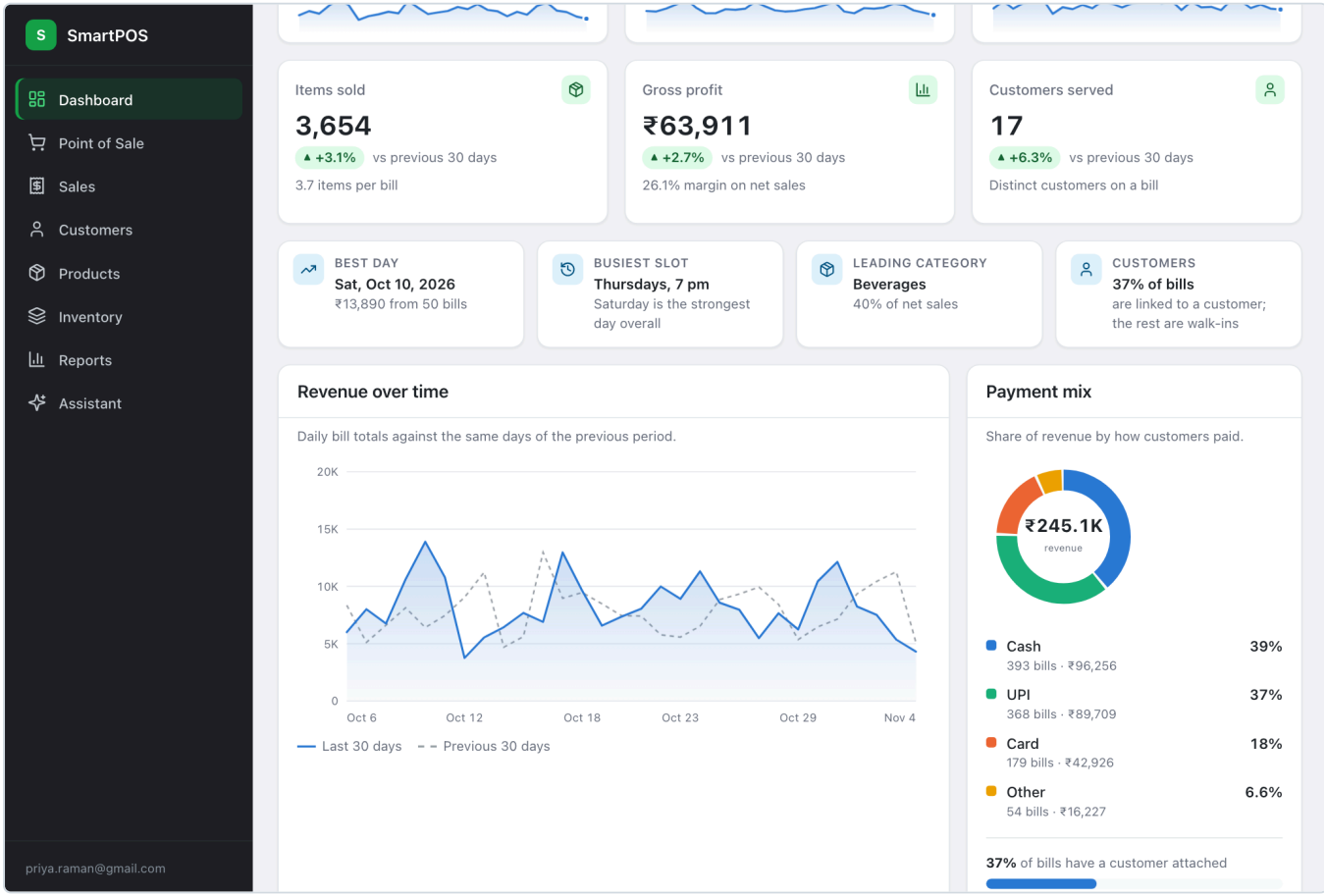


Figure 10.1. Sales performance for the last 30 days, each figure compared with the 30 days before

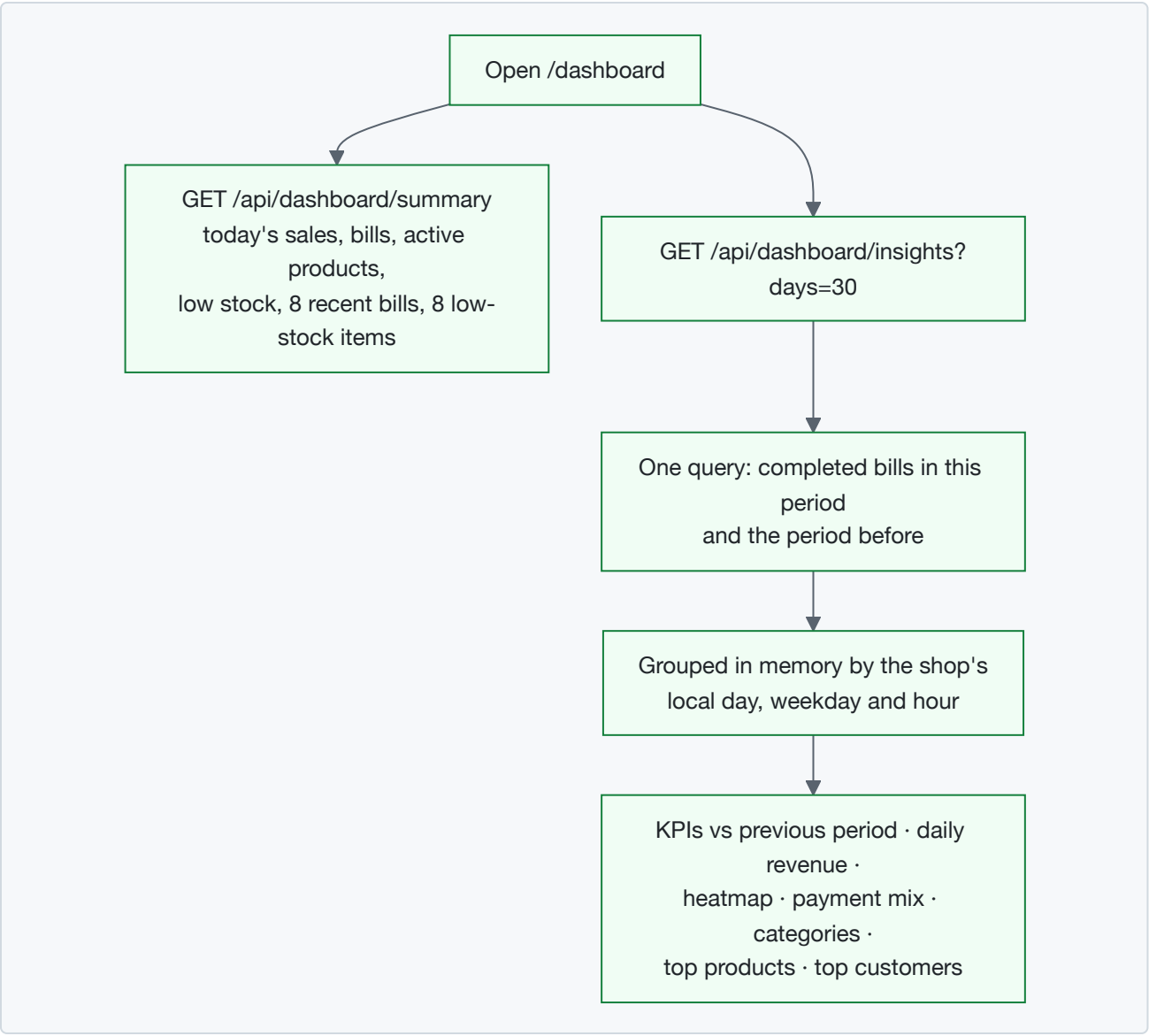


Figure 10.2. Where the dashboard numbers come from

Figure	Definition
Revenue	sum of completed bill totals
Average bill	revenue ÷ bills
Gross profit	Σ line totals − Σ (unit cost × quantity), using the cost recorded on each sale line
Customers served	distinct customers attached to a bill
Low stock	active products with stock ≤ their low-stock level

Only **Completed** bills count. Days, weekdays and hours are in the shop's time zone. The dashboard also holds the AI assistant panel (section 17). Tested: today's figures for the test shop were ₹4,313.50 from 22 bills, 16 active products and 3 low-stock.

11 Products and categories

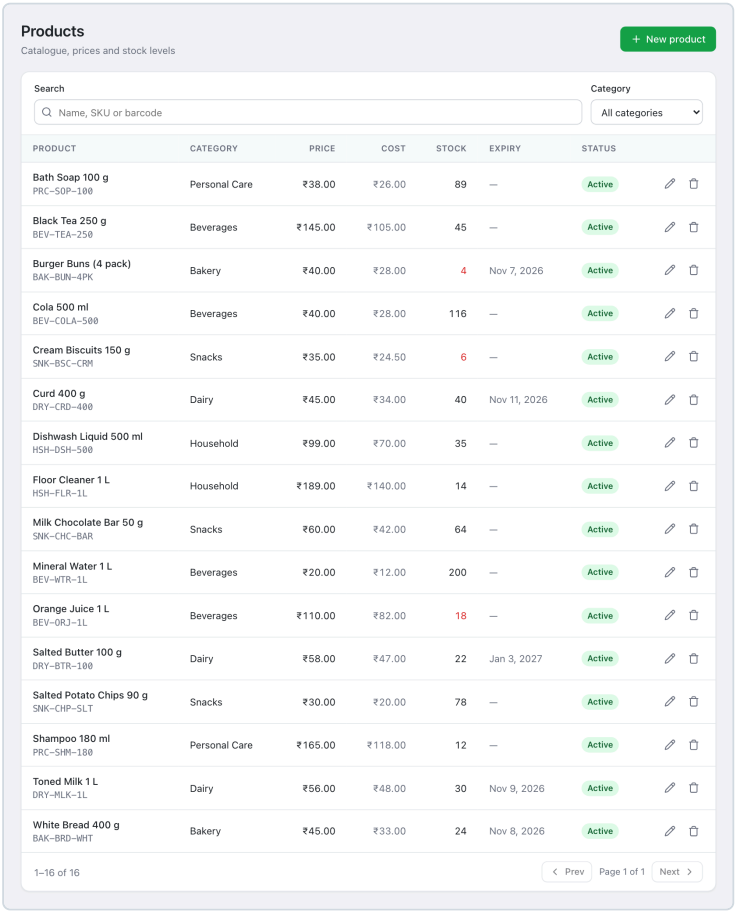


Figure 11.1. The catalogue: search by name, SKU or barcode; filter by category; stock shown red when low or out

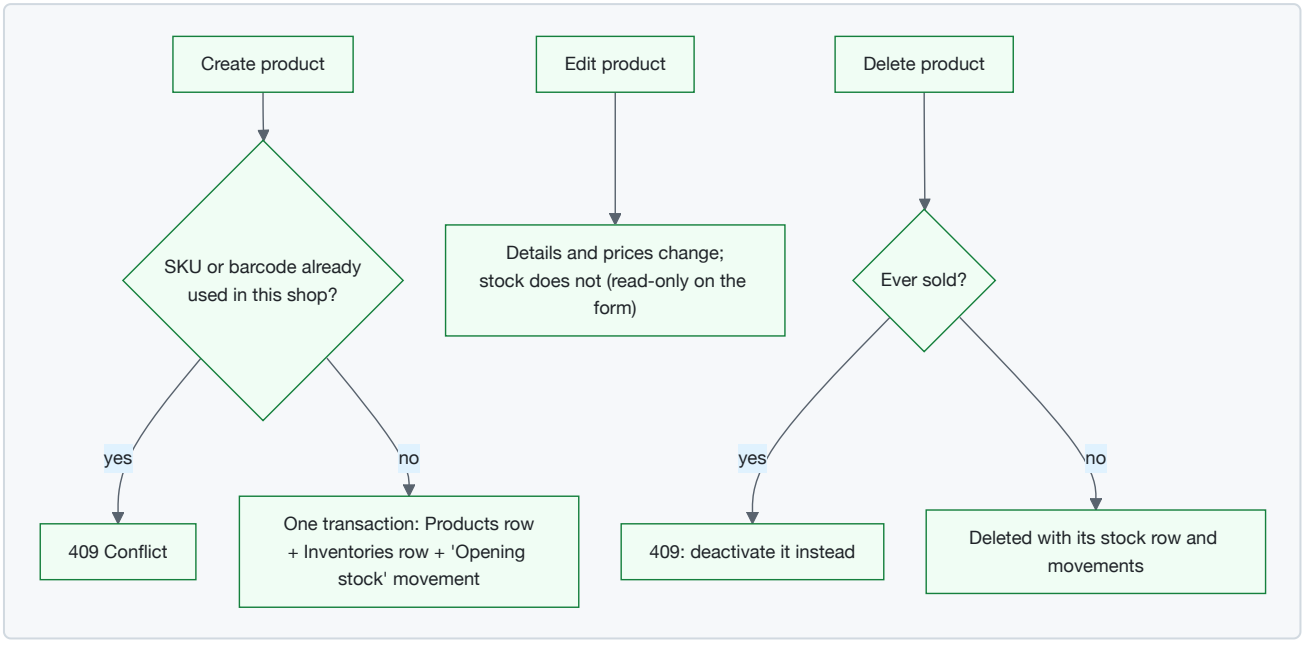


Figure 11.2. Product rules (ProductService)

- **Categories** are plain text. `GET /api/categories` lists the names in use with counts, for the filter and for suggestions while typing.

- **Inactive** products disappear from the POS but keep all their history.
- A separate document, *SmartPOS-Add-Product.pdf*, follows adding a product in full detail.

12 Point of Sale and checkout

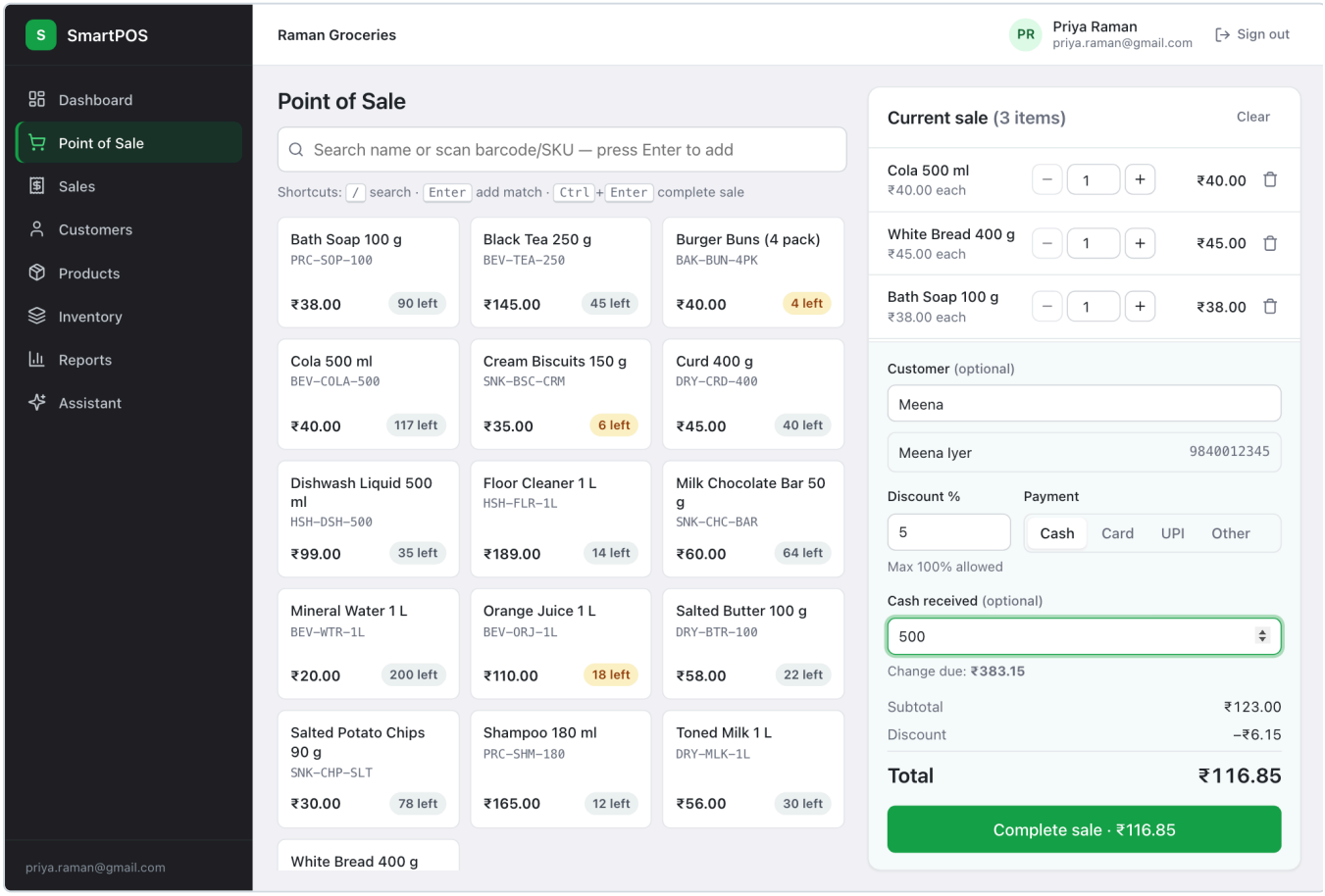


Figure 12.1. A bill being rung up: 4 items, customer "Meena Iyer", 5% discount, ₹500 cash received, so change due is ₹383.15

12.1 On the screen

- **Find products** by typing a name, SKU or barcode. A scanner works as "type + Enter": Enter adds an exact SKU/barcode match, or the only result. Shortcuts: `/` focuses the search, `Ctrl/⌘ + Enter` completes the sale.
- **The cart** can't take more than the stock on hand; tiles show "N left", and out-of-stock tiles are disabled.
- **Totals preview** uses the same arithmetic as the server (in whole cents), but the server recalculates everything from database prices.
- **Customer**: search by name or phone (2+ characters); the screen shows the points this bill will earn.
- **Payment**: Cash, Card, UPI or Other. For cash, enter the amount received to see the change, or leave it empty for the exact amount.

12.2 What the server does

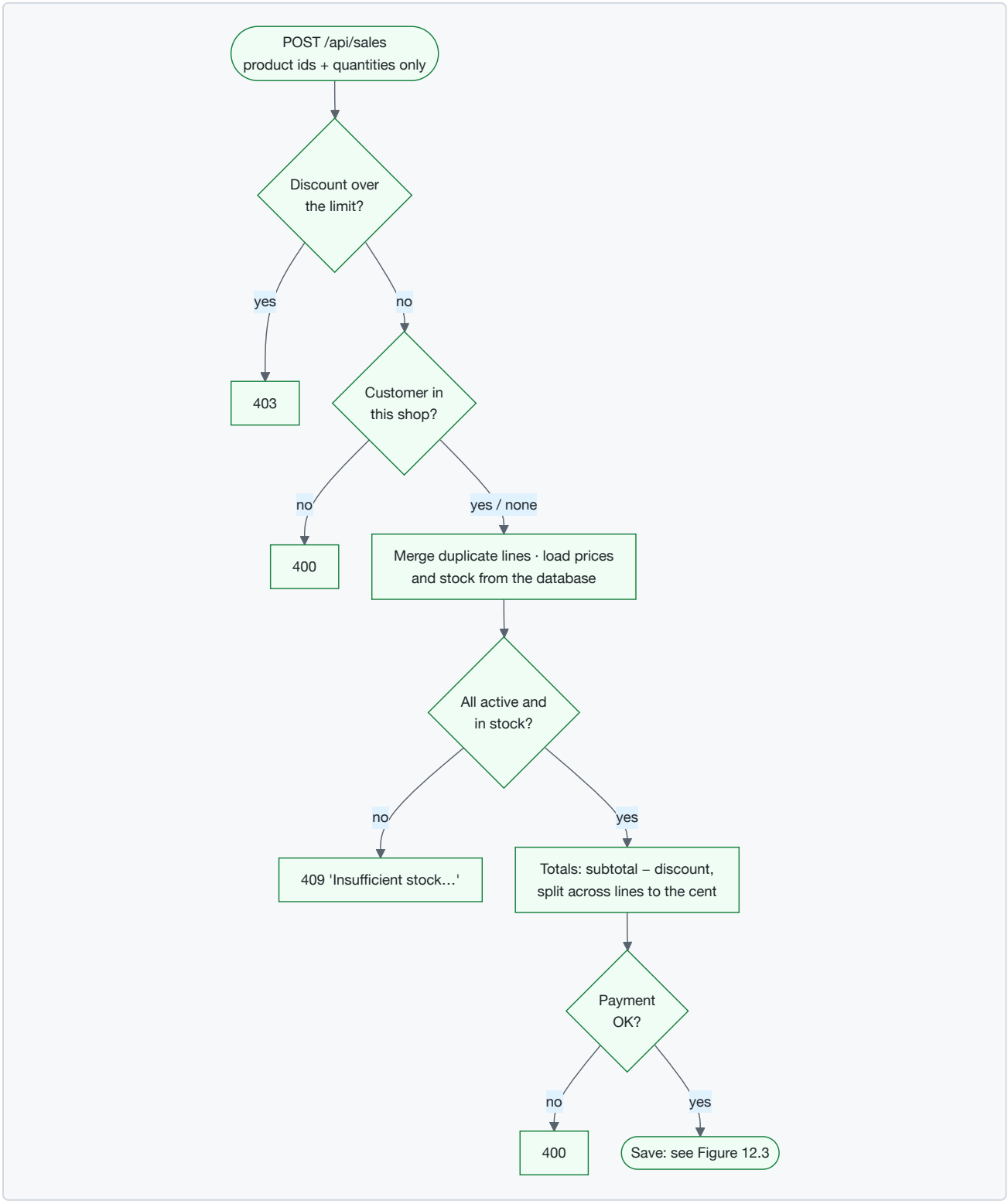


Figure 12.2. Checkout, part 1: the checks (SaleService.CreateAsync)

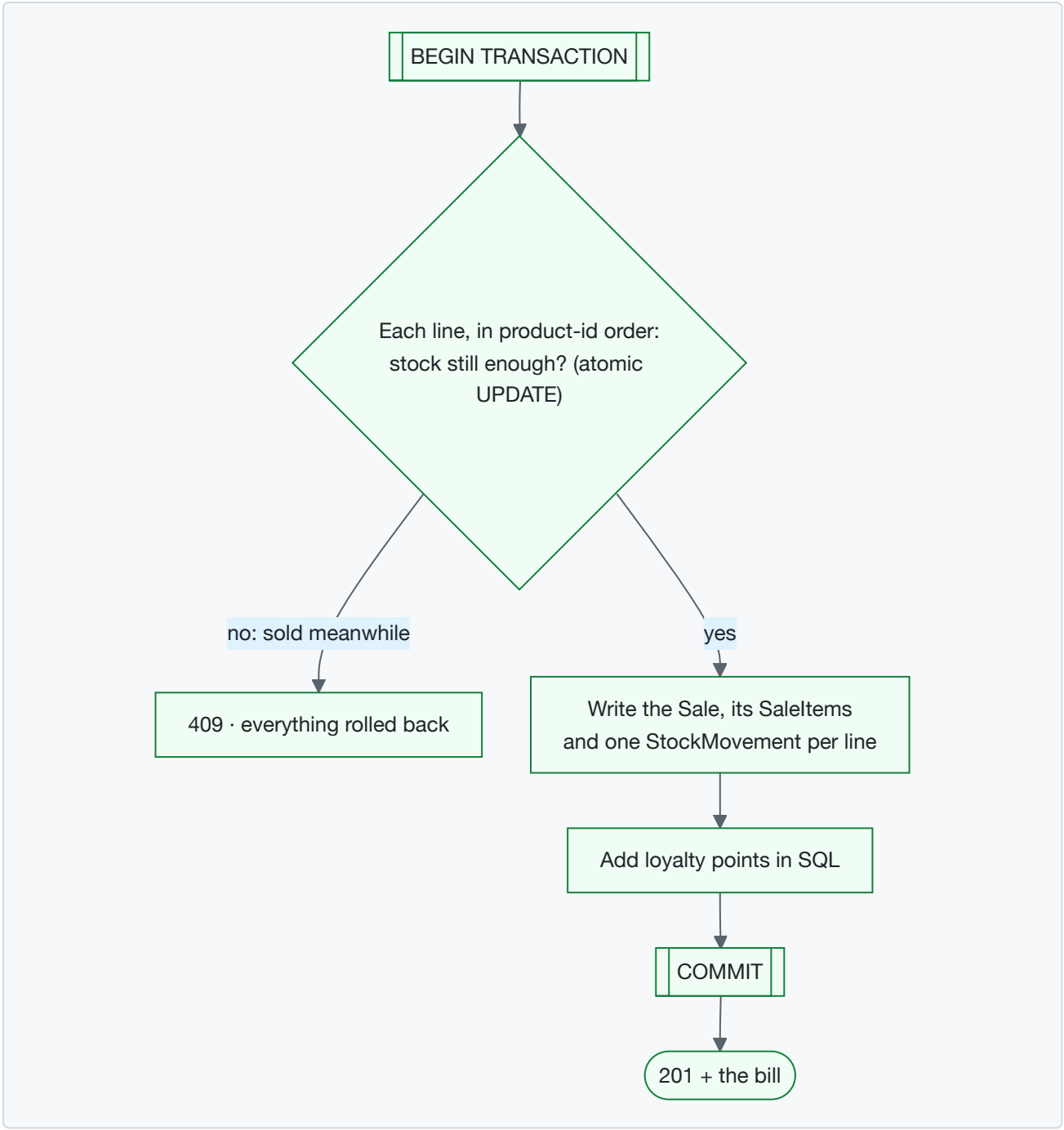


Figure 12.3. Checkout, part 2: saving (SaleService.CreateAsync)

12.3 The money, with a real bill

Step	Rule	Real bill BILL-20261104-89134635
Line subtotals	price × quantity	Cola 500 ml 3 × ₹40 = ₹120 · Potato Chips 2 × ₹30 = ₹60
Subtotal	sum of lines	₹180.00
Discount	subtotal × % (rounded, half away from zero)	10% → ₹18.00, split ₹12.00 + ₹6.00
Total	subtotal − discount (no tax)	**₹162.00** (lines ₹108.00 + ₹54.00)
Change	cash paid − total; other methods must be exact	₹200.00 paid → ₹38.00 change
Loyalty	floor(total ÷ 100) for a customer bill	1 point to Meena Iyer

WHY CHECKOUT IS SAFE

- **Prices come from the database**, so a tampered request can't change what the customer pays.
- **Two tills can't sell the last unit.** Stock is taken with one statement, `UPDATE … SET StockQuantity = StockQuantity − n WHERE StockQuantity >= n`; if it changes no row, the sale stops and everything is rolled back. The database's CHECK (stock ≥ 0) backs it up.
- **No deadlocks**: lines are processed in product-id order on every till.
- **Points can't be lost**: they are added in SQL (`LoyaltyPoints = LoyaltyPoints + n`), not read and rewritten.

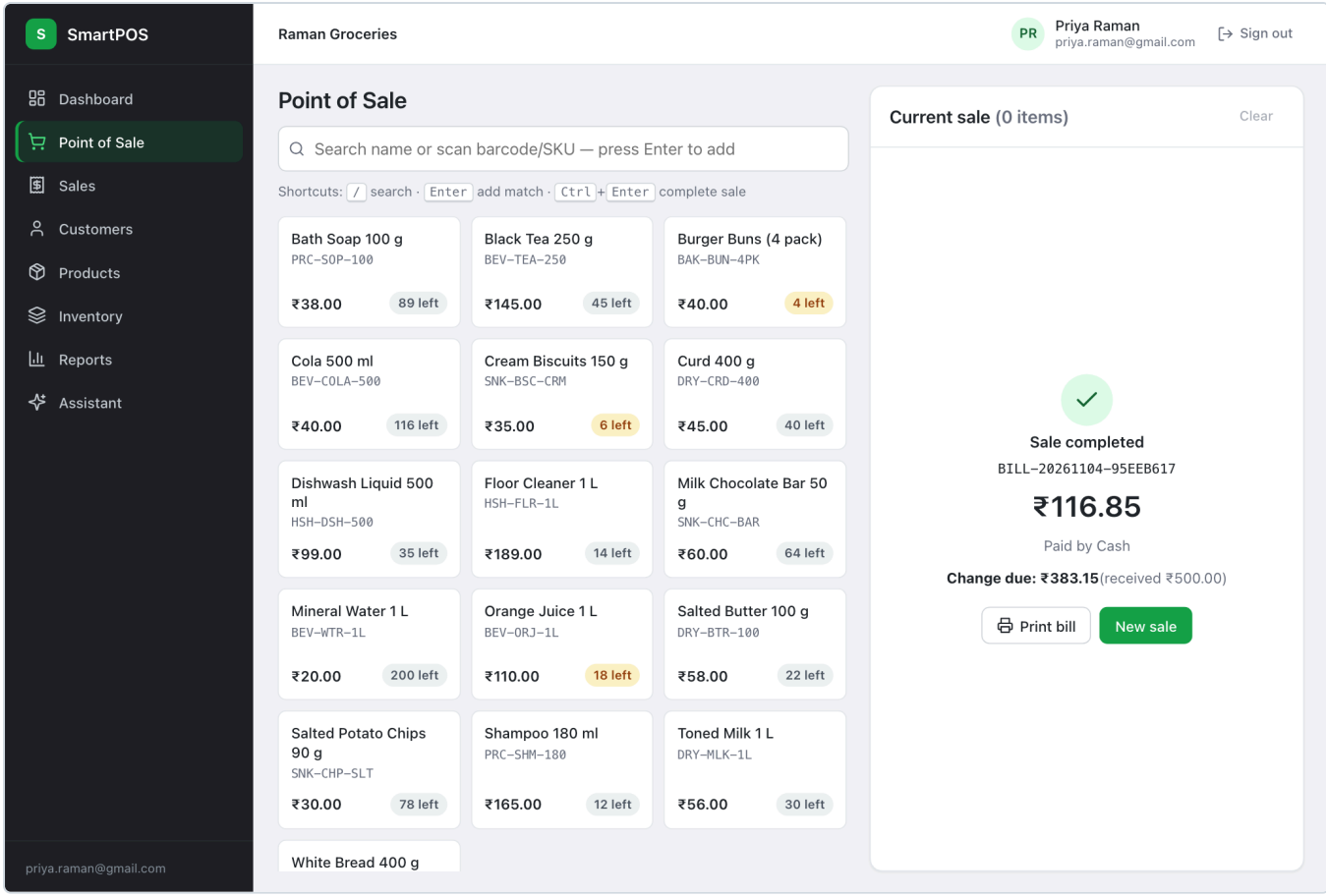


Figure 12.4. After checkout: bill number, total, change due, and buttons to print the bill or start a new sale

Tables: reads `Products`, `Inventories`, `Customers`; writes `Inventories`, `Sales`, `SaleItems`, `StockMovements`, `Customers`. **Code:** `SalesController`, `SaleService`, `SaleCalculator`, `StockRules`, `DiscountPolicy`, `LoyaltyRules`, `SmartPosDbContext.TryChangeStockAsync`; web `features/pos/*`.

13 Sales history and invoices

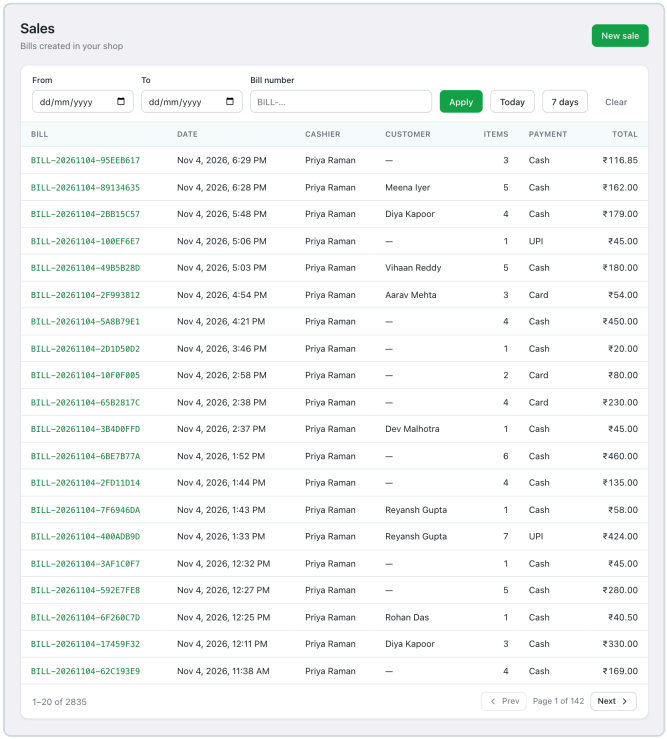


Figure 13.1. Sales: filter by date (Today, 7 days) or bill number, newest first

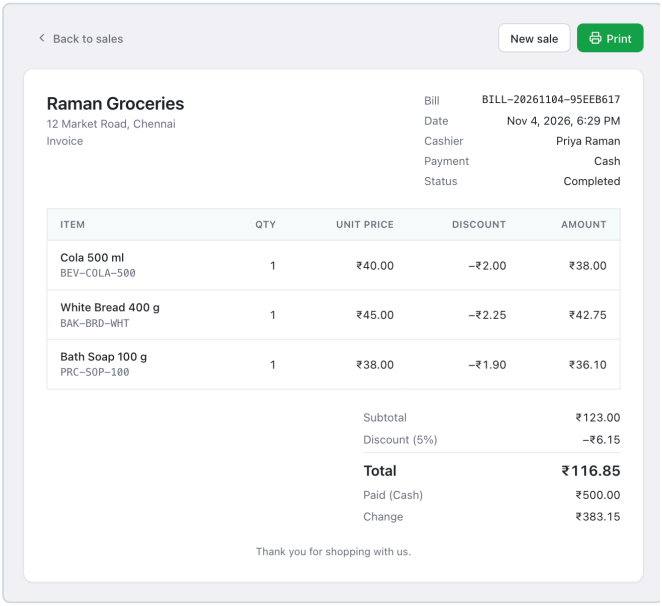


Figure 13.2. The invoice: each line with its share of the discount, the totals, payment and change. "Print" gives a clean printout.

- Dates are picked as the shop's local days and converted to a UTC range on the server: `[from 00:00, to + 1 day 00:00)`.
- An invoice shows the **snapshot** on each line, so it never changes after the sale.
- A bill from another shop is **404**, so ids can't be probed.
- There is no refund or cancel yet. `Status` already allows Cancelled, and reports already ignore anything but Completed.

14 Customers and loyalty points

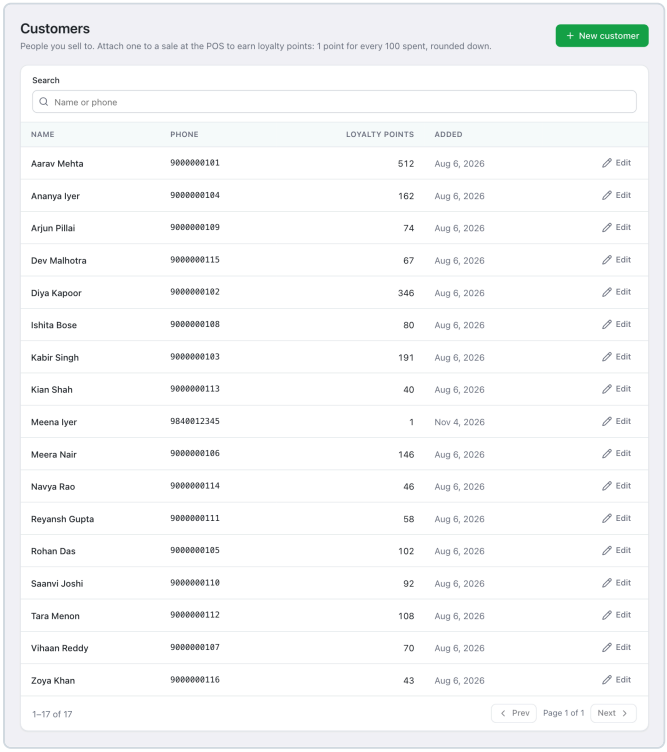


Figure 14.1. Customers with their points balance

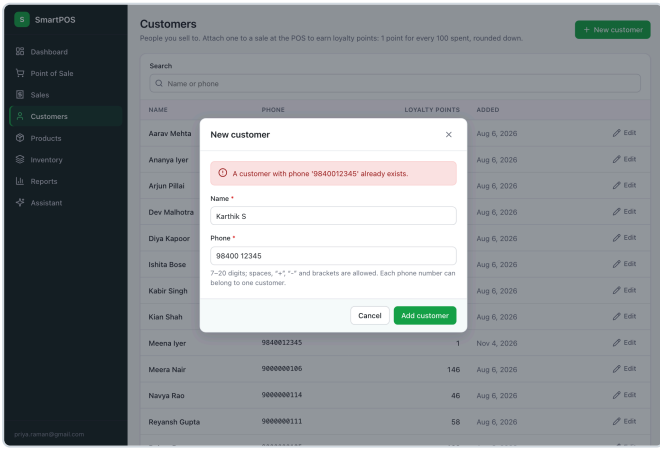


Figure 14.2. Typing "98400 12345" for a new customer: the server normalises it to 9840012345 and finds it already used (409)

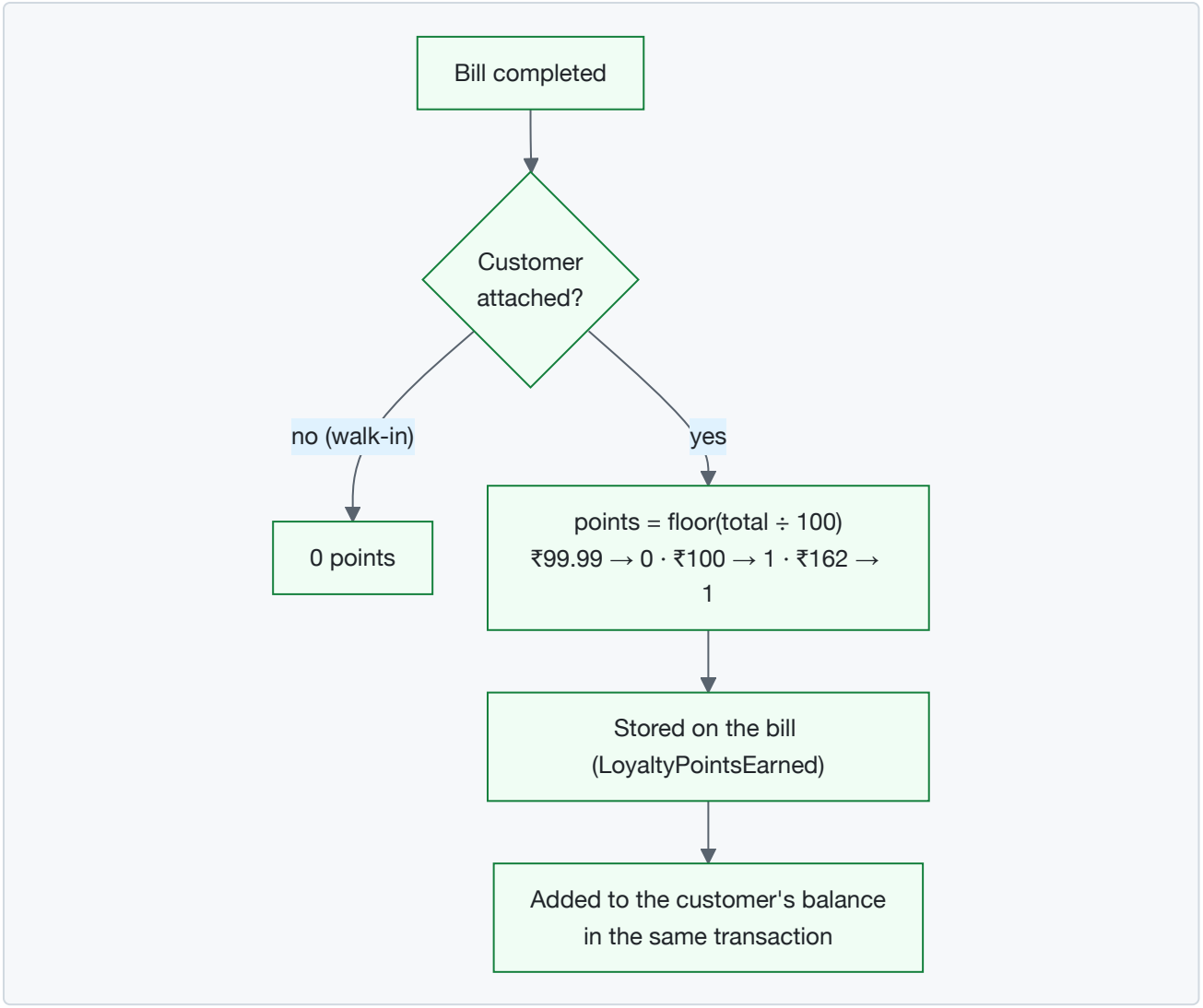


Figure 14.3. Earning loyalty points (LoyaltyRules)

The rate is `Loyalty:SpendPerPoint` (default 100). Points are always rounded down. A customer's balance always equals the sum of their bills' points. Points can't be spent yet (no redemption).

15 Inventory and stock movements

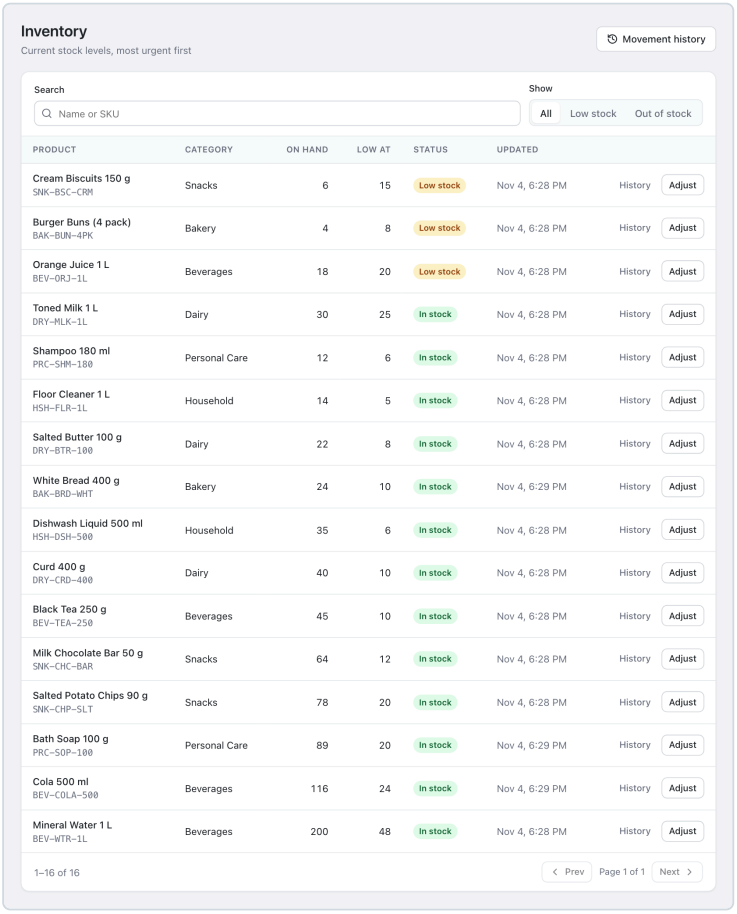


Figure 15.1. Inventory, most urgent first (furthest below its low-stock level), with All / Low stock / Out of stock filters

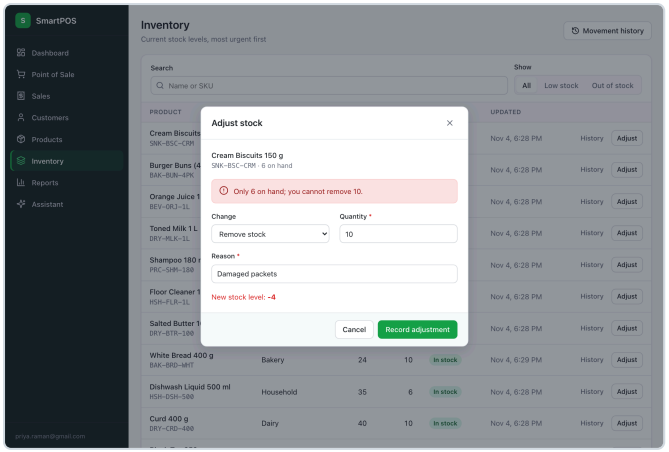


Figure 15.2. Removing 10 Cream Biscuits with only 6 on hand: refused in the browser, and "New stock level –4" is shown in red

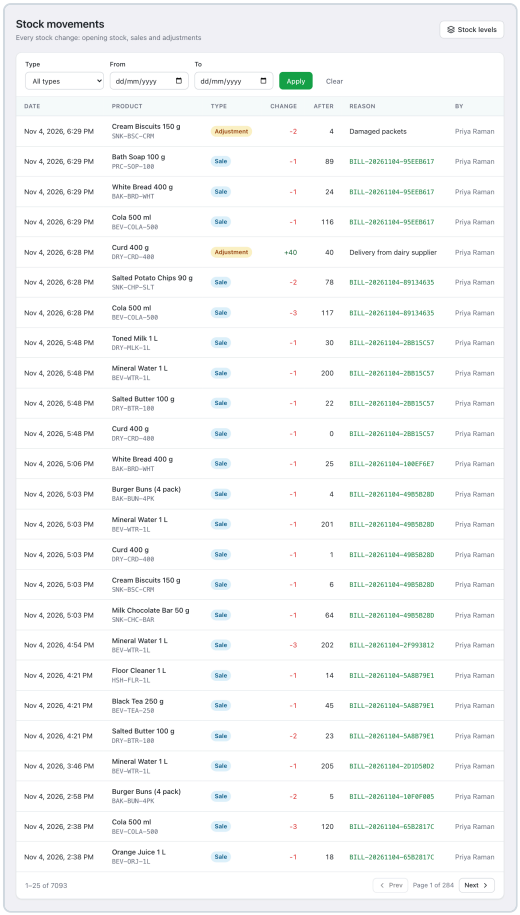


Figure 15.3. The movement history: every change, with its type, reason, bill and user

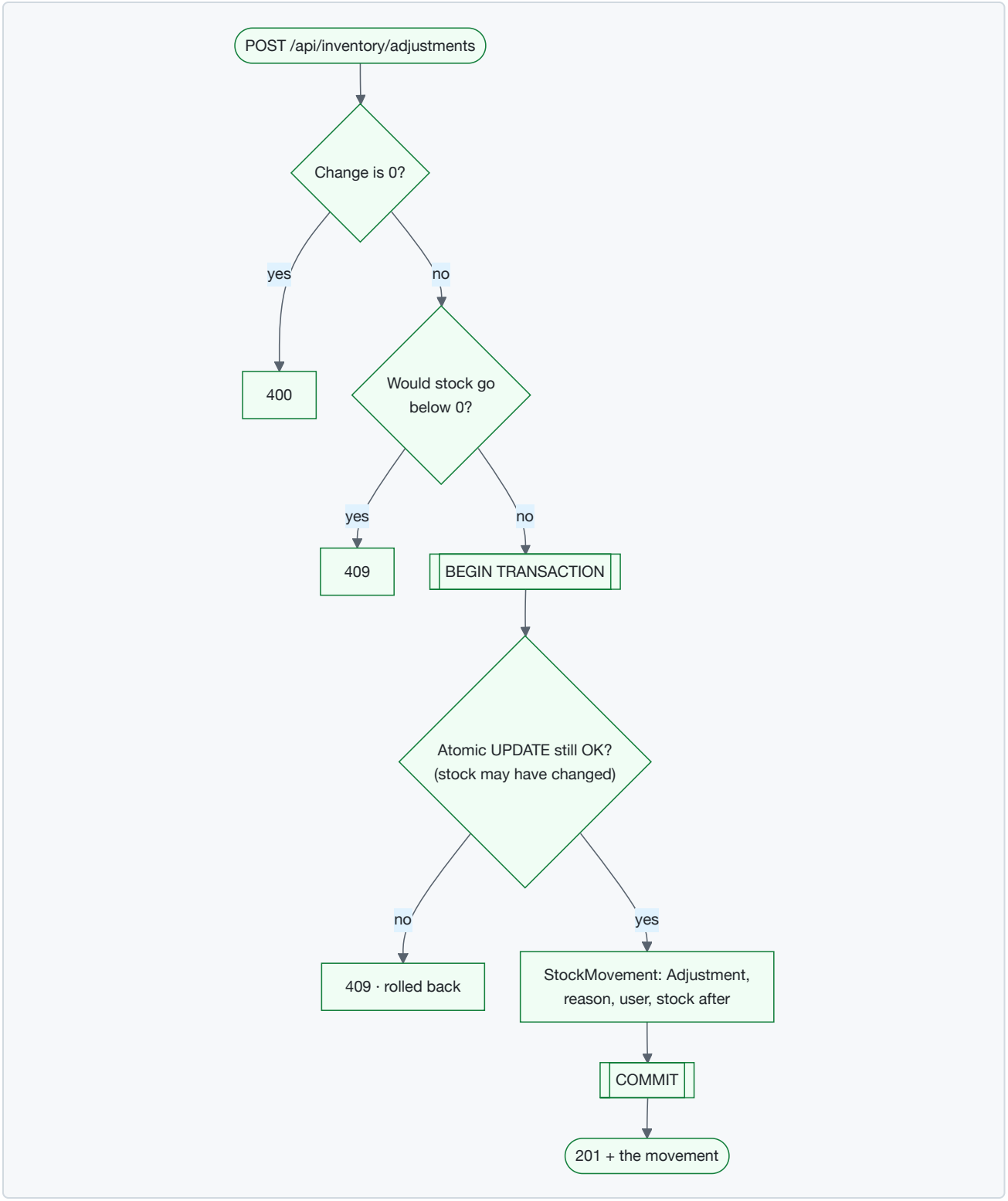


Figure 15.4. A stock adjustment (InventoryService.AdjustAsync)

Tested: adding 40 Curd ("Delivery from dairy supplier") took it from 0 to 40, recorded as an **Adjustment** by Priya Raman. The three ways stock changes (opening stock, sale, adjustment) all write a movement, so the ledger always adds up to the stock on hand.

16 Reports

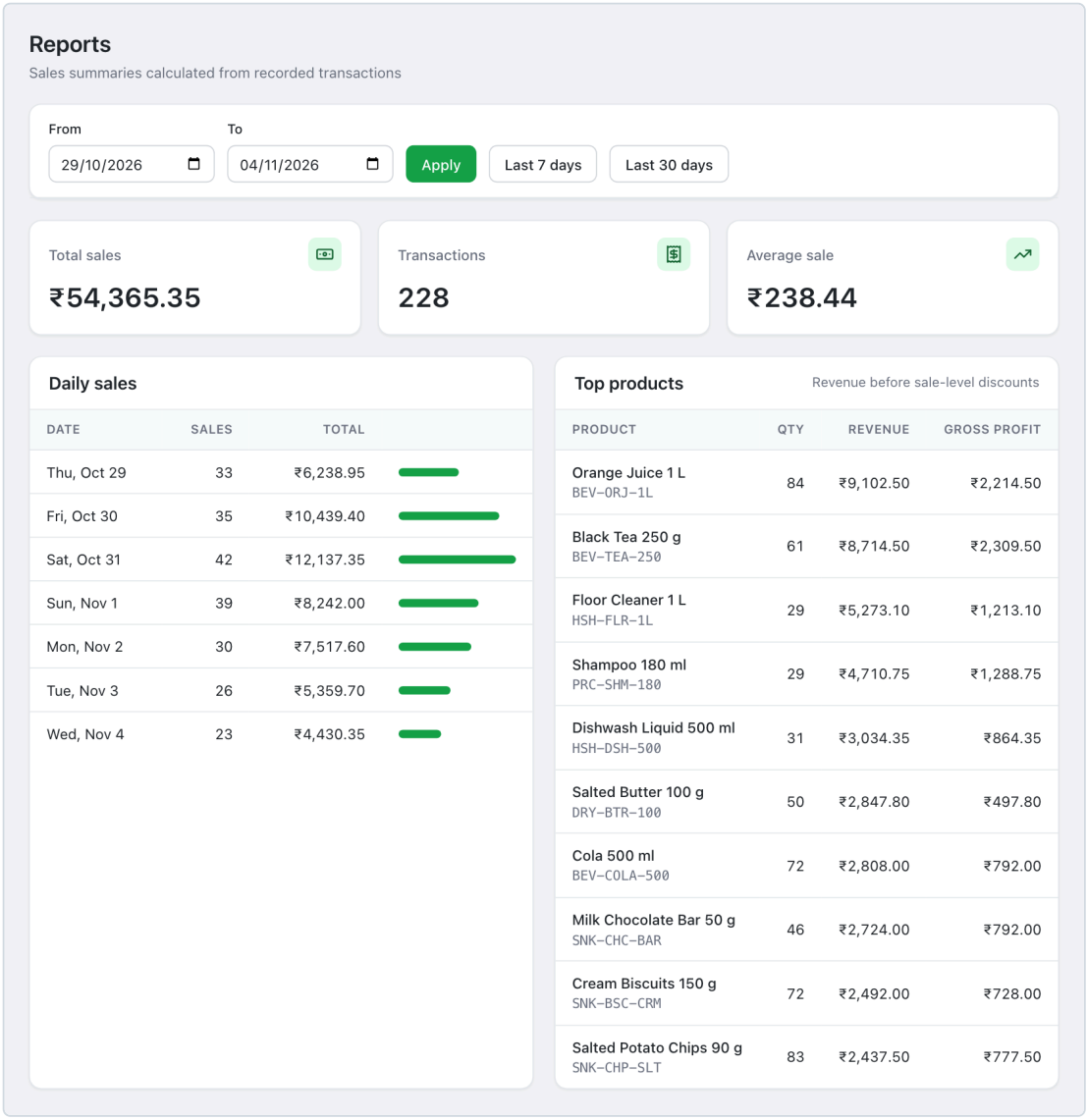


Figure 16.1. Reports for a date range: total, bills, average, daily sales with bars, and top products with revenue and gross profit

- **Daily sales** (`/api/reports/daily-sales`): every day in the range appears, even with no sales. It is grouped by the shop's local date, so the database doesn't need to know the time zone.
- **Top products** (`/api/reports/top-products`): by revenue (net sales after discount), with units, cost and gross profit from the snapshot costs.
- The AI assistant uses **these same queries**, so its figures always match the Reports page.

17 AI assistant

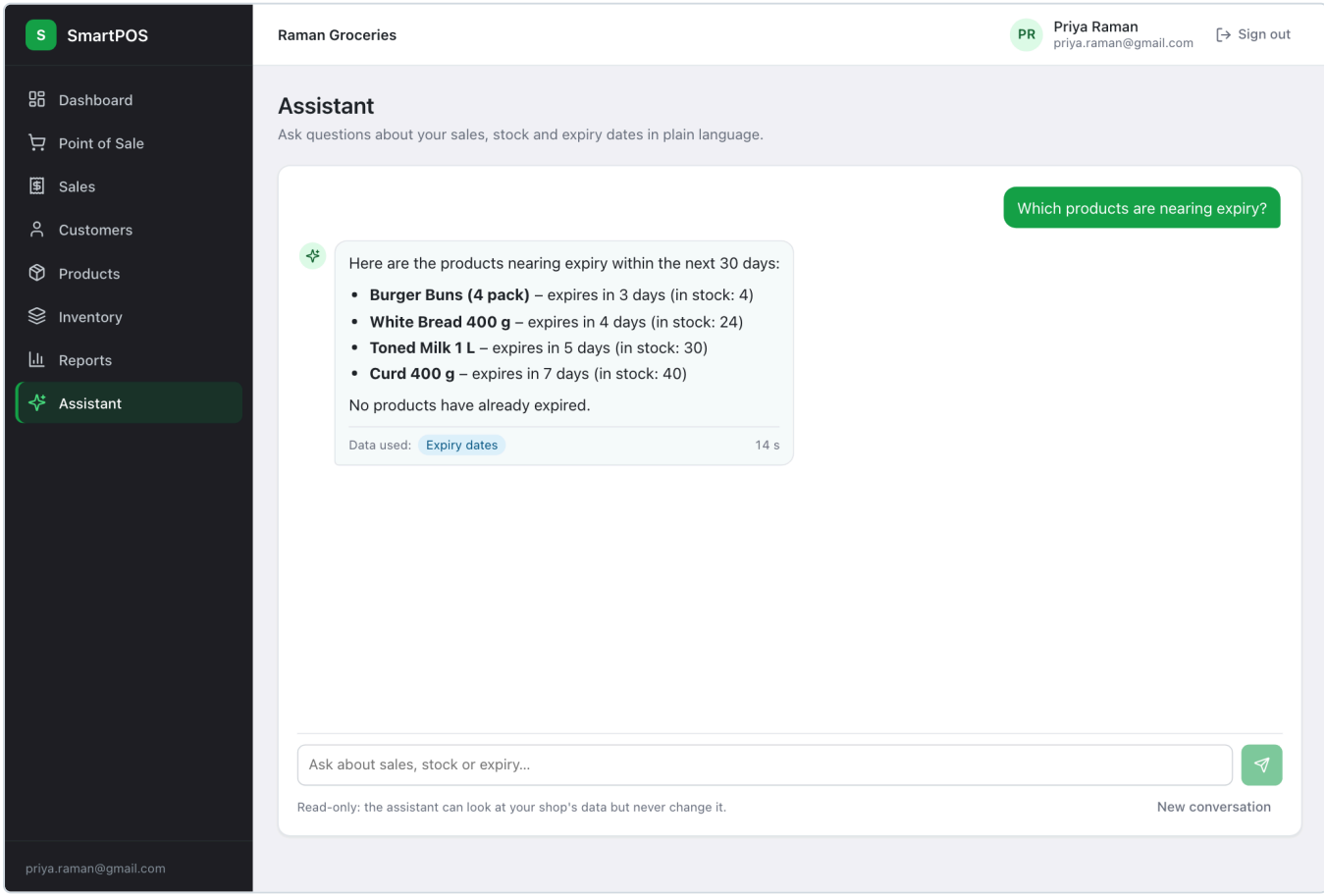


Figure 17.1. "Which products are nearing expiry?" The answer came from the shop's data, labelled "Data used: Expiry dates".

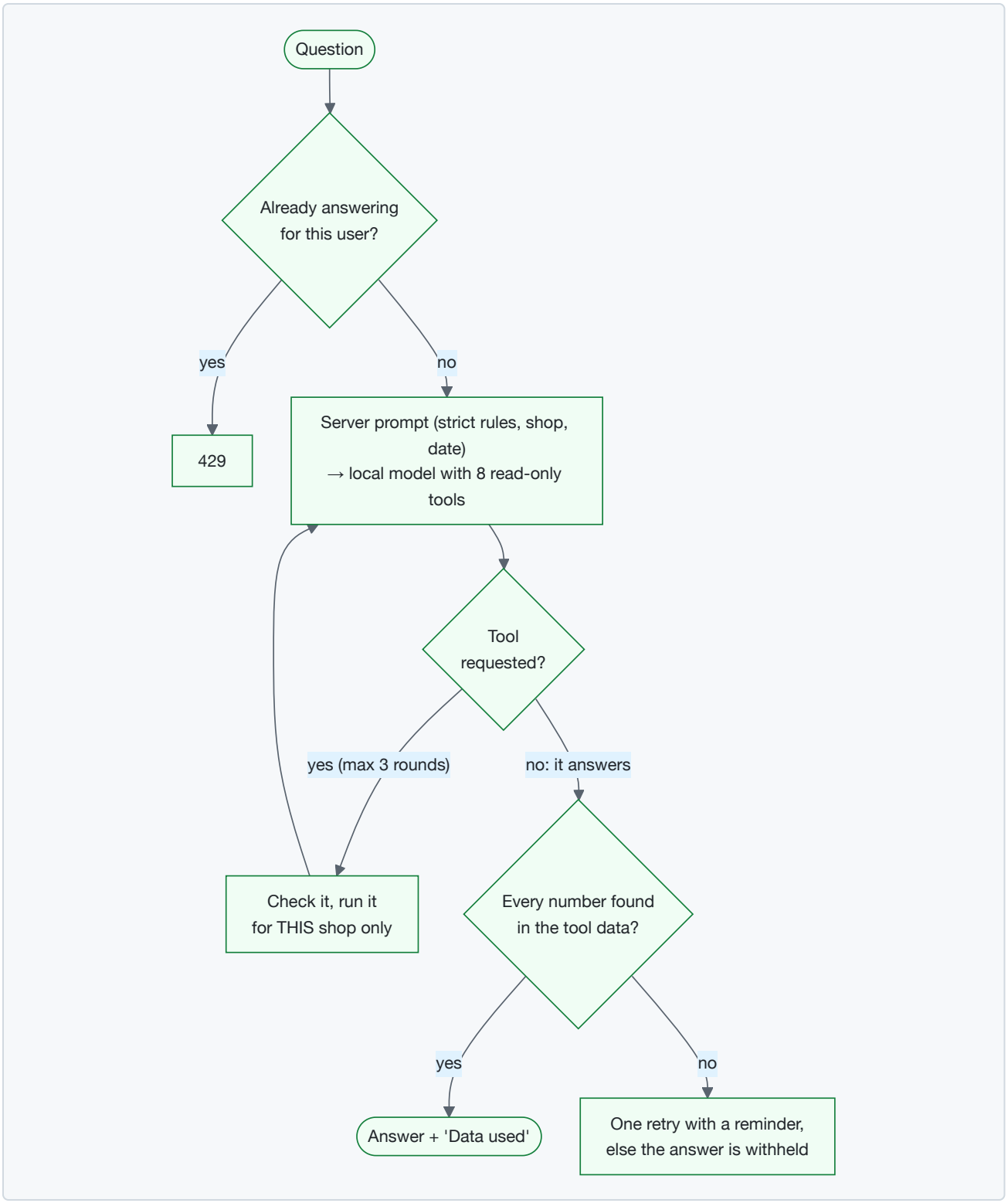


Figure 17.2. How a question is answered (AIOrchestrationService)

Safety rule	How it is enforced
Read-only	8 fixed tools that only read: today's/yesterday's sales, top sellers, most profitable, low stock, expiring, inventory value, unsold products. No free-form SQL.
Only your shop	No tool has a shop parameter; a made-up `shopId` argument is refused; the database filter applies
No invented figures	Strict prompt, plus a code check that every number appears in this question's tool data, with one retry before the answer is withheld

Safety rule	How it is enforced
Data stays on the computer	The model must be on localhost and not a cloud model; startup fails otherwise
Browser can't inject	History may only contain user/assistant turns; numbers in the history don't count as evidence
Privacy	Questions, answers and data are never logged (only timings and tool names)

Tested: "Which products are running low on stock?" called `GetLowStockProducts` and answered in 28.7 s with Cream Biscuits 150 g (6, level 15), Burger Buns (4, level 8) and Orange Juice 1 L (18, level 20), matching the inventory page.

PART E

Reference

Every endpoint, the settings, the known limitations, and how this document was checked.

18 API endpoints

Method + URL	What it does	Success
`POST /api/auth/register` · public	Create the user and shop, sign in	201
`POST /api/auth/login` · public	Sign in, get a token	200
`POST /api/auth/verify-email` · public	Confirm the email with the emailed token	204
`POST /api/auth/resend-verification`	Send a new link	204
`GET /api/auth/me`	The signed-in user and shop	200
`GET /api/settings/pos`	Shop name, address, currency, max discount, loyalty rate	200
`GET/POST /api/products`, `GET/PUT/DELETE /api/products/{id}`	Catalogue	200 / 201 / 204
`GET /api/categories`	Category names with counts	200
`GET/POST /api/customers`, `GET/PUT /api/customers/{id}`	Customers	200 / 201
`GET /api/inventory`	Stock levels (filter All / Low / OutOfStock)	200
`POST /api/inventory/adjustments`	Manual stock change with a reason	201
`GET /api/inventory/movements`	Stock ledger (by product, type, dates)	200
`POST /api/sales`	Checkout	201
`GET /api/sales`, `GET /api/sales/{id}`	Sales list, one invoice	200
`GET /api/dashboard/summary`, `/insights?days=`	Dashboard	200
`GET /api/reports/daily-sales`, `/top-products`	Reports	200
`POST /api/ai/chat`, `GET /api/ai/status`	Assistant	200
`GET /health` · public	Database and AI model status	200

Lists are paged: `?page=1&pageSize=20` (maximum 100), and they return `{ items, page, pageSize, totalCount, totalPages }`. Swagger documents every endpoint at `/swagger` in Development.

19 Settings

Setting	Default	Meaning
`ConnectionStrings:SmartPos`	(secret)	PostgreSQL connection; kept in user-secrets or an environment variable, never in a file
`Jwt:SigningKey` / `ExpiryMinutes`	(secret) / 480	Token signing key (≥ 32 characters) and lifetime
`Store:CurrencyCode` / `TimeZoneId`	INR / Asia/Kolkata	Money format and what "today" means
`Pos:MaxDiscountPercent`	100	Highest discount on a bill
`Loyalty:SpendPerPoint`	100	Spend per loyalty point
`RateLimiting:LoginPermitsPerMinute`	10	Sign-in/register attempts per IP per minute
`Ollama:*`, `Assistant:*`	localhost:11434, qwen3.5:9b; 10 questions/min	The AI model and its limits
`App:WebBaseUrl`, `Email:Smtp:*`	(Development: localhost:4200; log emails)	Links in emails and the mail server
`Database:SeedSampleData` / `SeedDemoHistory`	on in Development only	Sample catalogue and demo sales for new shops

20 Things to know and limitations

FOUND WHILE TESTING

- **A product created through the API without a price is stored at ₹0.00.** The web form requires prices, so this only affects direct API calls.
- **An expiry date in the past is accepted** when creating a product (this may be intended, for recording stock that has already expired).

NOT BUILT YET

- No refunds or cancelled bills; no staff accounts or permissions (one owner per shop); no tax; sale-level discounts only.
- Loyalty points can't be spent yet; email verification isn't enforced; there is no self-service password reset (a developer command exists).
- The sign-in token is kept in browser storage; an HTTP-only cookie would reduce the risk from malicious scripts.

21 How this document was checked

Everything was taken from the code and then confirmed against the running system on a **throwaway database**, never the real shop data:

- Two test shops were registered, getting the sample catalogue and 90 days of demo sales (5,757 bills, 14,330 sale lines and 14,364 stock movements across both shops).
- Every error message quoted came from a real request: registration, sign-in, customers, sales, adjustments, reports, the assistant, and cross-shop access.
- The bill in section 12.3 was created for this document and its arithmetic was checked by hand.
- Email verification was tested with the link the server produced: it works once.
- Every screenshot comes from the real web app, signed in as the test shop owner.
- Automated tests at the time of writing: 125 back-end unit tests, 86 integration tests and 40 web-app tests, all passing.